WNDRVR
SUPPORT NETWORK

VXWORKS 7 PPP PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 PPP Programmer's Guide, SR0650

10 June 2020

1. OVERVIEW

1. PPP for VxWorks 7

The VxWorks® 7 PPP Programmer's Guide describes the use of Point-to-Point Protocol (PPP), which allows the establishment, authentication, and control of PPP connections and PPP over Ethernet (PPPoE) implementations.

PPP provides a standard method for transporting multi-protocol datagrams over point-to-point links.

PPP is designed for simple links that transport packets between two peers. These links provide full-duplex, simultaneous, bidirectional operation, and are assumed to deliver packets in order.

Point-to-Point Protocol is capable of operating across most DTE/DCE interfaces, such as EIA RS-232-E, EIA RS-422, and CCITT V.35. The only absolute requirement imposed by PPP is the provision of a full-duplex circuit, either dedicated or circuit-switched, that can operate in asynchronous (start/stop), bit-synchronous, or octet-synchronous mode, transparent to PPP Data Link Layer frames.

PPPoE provides the ability to connect a network of hosts over a simple bridging device to a remote access concentrator. In this model, each host uses its own PPP stack, so each user is presented with a familiar user interface.

Access control, billing, and type of service can be managed on a per-user, rather than a per-site, basis.

To provide a point-to-point connection over Ethernet, each PPP session must learn the Ethernet address of the remote peer, as well as establish a unique session identifier. PPPoE defines a protocol that can be used to learn the Ethernet address of servers that provide PPPoE services.

Source Code Files

The source code files for PPP are in the following directory:

installDir **/vxworks-7/pkgs_v2/net/ipnet/linkproto/ppp-***x.x.x.x***/**

Next topic: Supported RFCs for PPP on page 1

2. Supported RFCs for PPP

Point-to-Point Protocol (PPP) request for comments (RFCs) contain standards and design information defined by the Internet Engineering Task Force (IETF).

Point-to-Point Protocol complies with the following IETF RFCs:

- RFC 1144, *Compressing TCP/IP Headers for Low-Speed Serial Links*
- RFC 1321, *The MD5 Message-Digest Algorithm*
- RFC 1332, *The PPP Internet Protocol Control Protocol*
  - Section 3.2: *IP-Compression-Protocol*is not implemented.
- RFC 1334, *PPP Authentication Protocols*
- RFC 1661, *The Point-to-Point Protocol*
  - Section 6.3: *Quality-Protocol* is not implemented.
- RFC 1662, *PPP in HDLC-like Framing*
- RFC 1877, *PPP IPCP Extensions for Name Server Addresses*

WNDRVR

- RFC 1990, *The PPP Multilink Protocol (MP)*
- RFC 1994, *Challenge/Handshake Authentication Protocol*
- RFC 2516, *A Method for Transmitting PPP over Ethernet (PPPoE)*
- RFC 5072, *IPv6 over PPP*

Previous topic: PPP for VxWorks 7 on page 1

2. PPP

1. About PPP Technology

Point-to-Point Protocol (PPP) can create a bidirectional link-layer network connection between two computers and allows both IPv4 and IPv6 over the connection.

PPP Client / Server Behavior

PPP is a peer-to-peer protocol, but it behaves as a client/server in the following ways:

- The peer that initiates the connection is considered the client, while the peer that waits for connection requests assumes a server role. A PPP server can wait passively, not sending packets until it receives a request from a client. To act as a server, the PPP_RUNMODE parameter must be set to passive. (For more information, see PPP Configuration Parameters on page 18.)
- A PPP server expects a client to authenticate itself in order to be granted access. The Password Authentication Protocol (PAP) and Challenge-Handshake Authentication Protocol (CHAP) facilitate authentication.

  Note: Your client and server environment provides a user management database for use with the rlogin, Telnet, SSH, and FTP server daemons. The environment also supports PPP PAP authentication and provides cryptography libraries for use with PPP CHAP authentication.

- A server can assign an IP address to a client if the client does not know which IP address to adopt. The server can also provide domain name server (DNS) addresses to the client.

Next topic: PPP Features on page 3

2. PPP Features

Point-to-Point Protocol (PPP) includes a number of features.

PPP Client

PPP can act as a PPP client and connect to PPP servers. The PPP client can optionally authenticate itself with the server using Password Authentication Protocol (PAP) or Challenge-Handshake Authentication Protocol (CHAP).

PPP Server

PPP can act as a PPP server and accept an unlimited number of connections from PPP clients. The clients can optionally be authenticated using PAP or CHAP.

PPPoE Client

PPP includes a PPPoE client implementation, capable of connecting to PPPoE servers.

PPPoE Server

PPP includes a PPPoE server implementation, which can accept any number of connections to PPPoE clients.

WNDRVR

PPP Shell Command

PPP includes a shell command in order to configure or list PPP and PPPoE interfaces.

Multiple Network Type Support

PPP can run under different network layers and includes built-in support for IPv4 and IPv6.

Multiple Driver Type Support

PPP can run over various network drivers, and it includes built-in support for serial drivers and for PPPoE with network interface drivers.

Multiple PPP Framing Support

PPP includes bit-synchronous and asynchronous high-level data link control (HDLC) framing and allows you to implement and add your own framing techniques.

Unlimited PPP Connections

PPP supports as many PPP connections as the system resources permit.

PPP Multilink Protocol (MP)

PPP supports the PPP Multilink protocol.

Van Jacobson Header Compression

PPP supports Van Jacobson header compression. Van Jacobson TCP/IP header compression (VJC) reduces the size of TCP/IP headers.

PPPoE over VLAN

PPP supports PPPoE over a Virtual LAN pseudo-interface.

Previous topic: About PPP Technology on page 3
Next topic: About PPP Configuration on page 4

3. About PPP Configuration

Each side of a Point-to-Point Protocol (PPP) connection has a list of required and/or desirable parameters for both incoming and outgoing communication. PPP has standard rules for the negotiation process that each side of the connection can accept, reject, or offer alternatives to parameter value proposals from its peer.

A link is initiated when one peer contacts the other. A connection is established when a peer negotiates successfully with the other and each side of the connection arrives at a mutually-agreeable local and remote configuration.

To facilitate and shorten the negotiation process, PPP assumes default values to most connection parameters.

Each protocol within a PPP stack manages a different aspect of the connection. To establish a link, one machine sends a Link Control Protocol (LCP) packet to its peer. If the peer is listening, it responds with LCP packets and begins negotiation of the link configuration parameters per RFC 1661.

After successfully completing authentication (if either peer requires it), the participants can exchange Network Control Protocol (NCP) packets to configure one or more network-layer protocols, such as IP over the PPP link.

PPP over Link Layers on page 5 You can establish a Point-to-Point Protocol (PPP) connection over a variety of link layers (communication media), such as a serial line over a null modem cable, a telephone connection over modems, or Ethernet. The link layer determines how PPP packets are encapsulated and framed.
PPP Connection Management on page 5 Each Point-to-Point Protocol (PPP) connection, and the state machine that governs the protocols in each connection, must be managed separately. Each PPP connection requires a separate network interface with a distinct remote address to distinguish it from other connections. PPP structures each PPP connection as a self-contained stack.

Previous topic: PPP Features on page 3
Next topic: About PPP Multilink Protocol (MP) on page 5

3.1. PPP over Link Layers

You can establish a Point-to-Point Protocol (PPP) connection over a variety of link layers (communication media), such as a serial line over a null modem cable, a telephone connection over modems, or Ethernet. The link layer determines how PPP packets are encapsulated and framed.

The choice of link layer may require that you configure physical devices. For example, computers connected over a null modem cable require the same bits per second (bps) setting for the serial device at each end, and computers connected over a modem must be configured to dial and/or answer calls.

The choice of link layer requires the software to perform certain handshaking before the PPP protocol starts. For instance, clients connect to a Windows host over a null modem connection with a non-standard handshaking process. A Windows client sends the string "CLIENT" and expects the server to respond with the string "CLIENTSERVER" prior to the start of PPP negotiation.

Parent topic: About PPP Configuration on page 4

3.2. PPP Connection Management

Each Point-to-Point Protocol (PPP) connection, and the state machine that governs the protocols in each connection, must be managed separately. Each PPP connection requires a separate network interface with a distinct remote address to distinguish it from other connections. PPP structures each PPP connection as a self-contained stack.

The purpose of connecting computers is to have a way to transmit and receive application data (for instance, IP datagrams) over a link. But in order to establish and maintain a link, the PPP control protocols (such as LCP, CHAP, IPCP, and so forth) must exchange control messages with the peer. Control messages are never seen at the IP level, but are intercepted by control protocols within the PPP stack. Conversely, data packets are not seen by control protocols. However, both control packets and data packets must be encapsulated as PPP packets and framed as determined by the underlying link layer.

Parent topic: About PPP Configuration on page 4

4. About PPP Multilink Protocol (MP)

Point-to-Point Protocol (PPP) supports the PPP Multilink Protocol (MP). In the PPP MP implementation, physical links are bundled to create a single interface that represents a single logical connection.

A bundle consists of multiple physical links, increasing the total throughput. PPP supports maximum size bundles of 10 links. The maximum size of a bundle is set by the constant **IPPPP_MAX_LINKS** in the **ipppp_config.h** header file.

PPP MP supports combining physical links with different types into bundles. A serial link can be joined to a bundle that was created with a PPPoE link, and vice-versa. PPPoE can start up multiple links and join them to a bundle as they come up. Serial interfaces have one link, based on the *devname* system variable associated with that interface. PPPoE interfaces, however, can open multiple links by starting new sessions with the same server, as well as bundle links from other PPPoE interfaces, up to the maximum bundle size (**IPPPP_MAX_LINKS**).

With serial links, there is a limit of one link per interface because the driver gets its device based on the interface name. However, PPPoE can start up new links by starting a new session with the server.

When a bundle is created, no new interface is created. When an interface negotiates MP, using the LCP option MRRU on a link, it becomes a bundle. Any link that comes up after that time, with the same peer, is joined to that bundle (if the MRRU values match, and the link passes authorization, if enabled).

The multilink connection, or bundle, appears as an interface but actually consists of one or more PPP interfaces. The bundle acts as a "virtual link." Each interface, or link, in a bundle can send and receive non-multilink packets, as well as Multilink packets. When a link in bundle goes down, it is automatically removed from the bundle.

Multilink bundles act as ordinary interfaces. The protocol manages the multiplexing of packets, fragmentation, and reassembling of packets.

When sending packets, the links in the bundle are used in rotation.

Build Time Configuration Summary - PPP Multilink on page 6 Point-to-Point Protocol (PPP) Multilink Protocol (MP) configuration summary.
Build Time Configuration Summary - Multilink with PPPoE on page 6 Point-to-Point Protocol over Ethernet (PPPoE) Multilink Protocol (MP) configuration summary.

Previous topic: About PPP Configuration on page 4
Next topic: PPPoE on page 7

4.1. Build Time Configuration Summary - PPP Multilink

Point-to-Point Protocol (PPP) Multilink Protocol (MP) configuration summary.

About This Task

To configure and build a VxWorks image that supports the PPP Multilink Protocol (MP), proceed as follows:

Procedure

1. Include the IPPPP_MP option in your VxWorks source build (VSB).
2. Build the VSB project.
3. Create a VxWorks image project (VIP) and include the INCLUDE_IPPPPMP component.
4. Optionally, set the PPPMP_MRRU parameter. See PPP Multilink Protocol Configuration Parameters on page 31.
5. Build the VIP.

Parent topic: About PPP Multilink Protocol (MP) on page 5

4.2. Build Time Configuration Summary - Multilink with PPPoE

Point-to-Point Protocol over Ethernet (PPPoE) Multilink Protocol (MP) configuration summary.

About This Task

To configure and build a VxWorks image that supports the multilink with PPPoE protocol, proceed as follows:

Procedure

1. Include the IPPPP_MP option in your VxWorks Source Build (VSB).
2. Build the VSB project.
3. Create a VxWorks image project (VIP) and include the INCLUDE_IPPPPMP and INCLUDE_IPPPPOE components.
4. Optionally, set the PPPMP_MRRU and PPPMP_PPPEO_LINKS parameters. See PPP Multilink Protocol Configuration Parameters on page 31.
5. Build the VIP.

Parent topic: About PPP Multilink Protocol (MP) on page 5

5. PPPoE

Wind River recommends that, if you are not already familiar with Point-to-Point Protocol over Ethernet (PPPoE), you should read RFC 2516 to get a basic understanding of PPPoE.

Modern access technologies are faced with several conflicting goals. It is desirable to connect multiple hosts at a remote site through the same access device on customer premises. It is also a goal to provide access control and billing functionality in a manner similar to dial-up services using PPP. In many access technologies, the most cost-effective method to attach multiple hosts to the customer on-site access device is using Ethernet. In addition, it is desirable to keep the cost of this device as low as possible, while requiring as little configuration as possible.

PPPoE allows several hosts to connect to a remote access concentrator. Each host uses its own PPP stack. Each PPP session must learn the Ethernet address of the remote peer to which it connects and establish a unique session identifier. PPPoE includes a discovery protocol that provides this.

The PPPoE Protocol

The PPPoE protocol uses six-byte headers sent in Ethernet frames with the MAC type set to either **0x8863** (discovery stage) or **0x8864** (PPP session stage). The PPPoE header consists of a one-byte version/type field, a one-byte code field, a two-byte session ID, and a two-byte length field.

The PPPoE protocol has two distinct stages: the discovery stage and the PPP session stage. When a host wishes to initiate a PPPoE session, it must first perform discovery to identify the Ethernet MAC address of the peer and establish a PPPoE session ID. While PPP defines a peer-to-peer relationship, discovery is inherently a client-server relationship.

In the discovery process, a host (the client) discovers an access concentrator (the server). Based on network topology, the host may be able to communicate with more than one access concentrator. The discovery stage allows the host to discover all access concentrators and then select one. When discovery completes successfully, both the host and the selected access concentrator have the information they need to build their point-to-point connection over Ethernet.

The discovery stage remains stateless until a PPP session is established. Once a PPP session is established, both the host and the access concentrator must allocate the resources for a PPP virtual interface.

PPPoE Discovery Stage Process on page 8 There are four steps to the Discovery stage. When it completes, both peers have acquired the Point-to-Point Protocol over Ethernet (PPPoE) session ID and the peer's Ethernet address, which together uniquely define the PPPoE session.

PPPoE PPP Session Stage on page 8 PPP sessions can be established following a successful discovery stage.

Previous topic: About PPP Multilink Protocol (MP) on page 5
Next topic: PPPoE Over VLAN on page 9

5.1. PPPoE Discovery Stage Process

There are four steps to the Discovery stage. When it completes, both peers have acquired the Point-to-Point Protocol over Ethernet (PPPoE) session ID and the peer's Ethernet address, which together uniquely define the PPPoE session.

These steps are as follows:

1. The host transmits a PPPoE Active Discovery Initiation (PADI).

   The host (the client) broadcasts a PADI packet to the local Ethernet LAN. This packet signifies the need for a PPP server, that is, the PPPoE access concentrator.

2. The Access Concentrator replies with a PPPoE Active Discovery Offer (PADO).

   If the access concentrator (the server) is willing to accept another PPP session, it replies with a PADO packet sent directly to the host, listing its available services.

3. The host sends a PPPoE Active Discovery Request (PADR) to the chosen access concentrator.

   After the host broadcasts the PADI, it can receive more than one PADO. The host looks through the PADO packets it receives and chooses one. The choice can be based on the access concentrator name, or on the available services offered. The host then sends one PADR packet to the access concentrator that it has chosen.

4. The access concentrator replies with a PPPoE Active Discovery Session-confirmation (PADS).

   When the access concentrator receives a PADR packet, it prepares to begin a PPP session. It generates a unique session ID for the PPPoE session and replies to the host with a PADS packet. Immediately after sending the PADS, the access concentrator proceeds to the PPP session stage. When the host receives the PADS packet, it also proceeds to the PPP session stage.

Note: The discovery stage remains stateless until after the fourth step, when the host has chosen an access concentrator and been accepted. The discovery stage is then completed and the session stage begins.

Parent topic: PPPoE on page 7

5.2. PPPoE PPP Session Stage

PPP sessions can be established following a successful discovery stage.

After the PPPoE Discovery Stage Process on page 8, the host and the access concentrator can transmit Point-to-Point Protocol (PPP) frames in PPPoE packets over Ethernet, just as in any other PPP encapsulation

All PPPoE session packets are sent unicast and use a reserved Ethernet frame. The PPPoE payload contains one unfragmented PPP frame each, which begins with the PPP Protocol-ID.

WNDRVR

Note: The maximum transmission unit (MTU) and maximum receive unit (MRU) of a PPPoE interface are both 1492 bytes (because the PPPoE header is six bytes long.)

Parent topic: PPPoE on page 7

6. PPPoE Over VLAN

Point-to-Point Protocol over Ethernet (PPPoE) over VLAN is supported on a VLAN pseudo-interface not a VLAN socket interface.

PPPoE-over-VLAN and PPPoE Connections on the Same Client on page 9 You can connect both a client and server by Ethernet, VLAN and Point-to-Point Protocol over Ethernet (PPOE) over a VLAN.
Clients on the Same VLAN and Clients Determine Addresses on page 10 You can have a Point-to-Point Protocol over Ethernet (PPOE) over VLAN connection between two clients and a server where the clients determine the PPP IP addresses.
Clients on the Same VLAN and Server Determines Addresses on page 11 You can have a Point-to-Point Protocol over Ethernet (PPOE) over VLAN connection between two clients and a server where the server determines the PPP IP addresses.
Clients on Different VLANs and the Server Determines Addresses on page 11 The server can determine IP addresses for clients of different VLANs.
Build-Time Configuration Summary for the Server and the Client on page 12 Configuring and building a VxWorks image that supports the PPPoE over VLAN protocol is the same as doing it for Point-to-Point Protocol over Ethernet (PPPoE).
Run-Time Configuration Summary for the PPPoE Server on page 13 How to configure the Point-to-Point Protocol over Ethernet (PPPoE) server at runtime.
Run-Time Configuration Summary for the Client on page 13 How to configure the Point-to-Point Protocol over Ethernet (PPPoE) client at runtime.

Previous topic: PPPoE on page 7
Next topic: Van Jacobson Header Compression on page 13

6.1. PPPoE-over-VLAN and PPPoE Connections on the Same Client

You can connect both a client and server by Ethernet, VLAN and Point-to-Point Protocol over Ethernet (PPOE) over a VLAN.

The figure below illustrates a PPPoE over VLAN connection and a PPPoE connection between a client and a server. Both nodes reside on the same LAN. Both nodes have a VLAN interface that has been assigned the same VLAN tag. The parent physical interface for the VLAN interface on each node is the physical interface to the LAN. A PPPoE interface is created on the client using a console command and is associated with the VLAN interface using another console command. The PPPoE-over-VLAN interface is put into service using a console command, and this action triggers the sending of a connection request to the server. The exchange of control packets between the client and the server results in a PPPoE interface being established on the server, and this interface uses the VLAN interface, and indirectly the Ethernet interface, for transport.

Another PPPoE interface is created on the client and is associated with the Ethernet interface. Putting this interface into service causes an exchange of control packets between the server and the client, which results in a PPPoE interface being established on the server, and this interface uses the Ethernet interface directly for transport.

For an overview of the run-time configuration, see Run-Time Configuration Summary for the PPPoE Server on page 13 and Run-Time Configuration Summary for the Client on page 13.

The IP addresses used by the server and client on the PPPoE-over-VLAN connection and the PPPoE connection are negotiated during the Internet Protocol Control Protocol (IPCP) phase of the PPP call setup. The IPCP IP addresses shown for the client and the server in the figure below are the result of applying the pattern shown in *Case 1* or *2* in the table *Configuring IPCP IP Address Negotiation Between Two Nodes* in PPP Configuration Parameters on page 18. The local address is set statically in the server, and the client requests an address from the server and the server provides an address from an address pool.

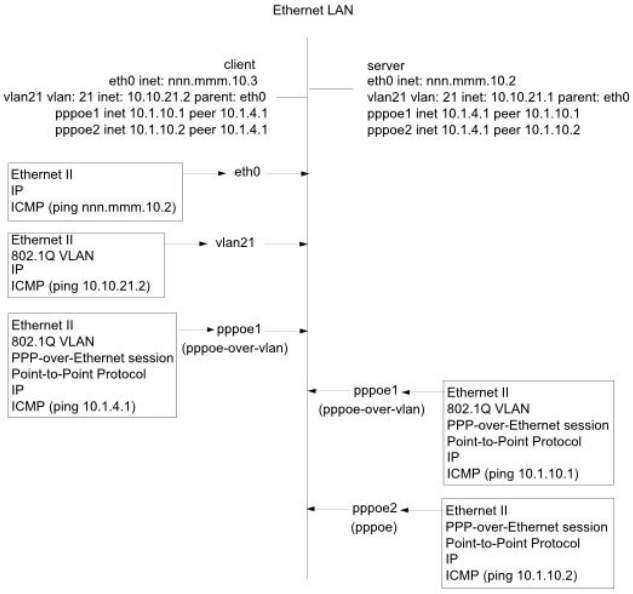
The figure below shows the client pinging the server using the Ethernet, VLAN and PPPoE-over-VLAN interfaces. When the client pings the server's public IP address, the ping request is sent to the Ethernet interface (eth0). When the client pings the server's VLAN

IP address, the ping request is sent to the VLAN pseudo-interface (vlan21) and then to the Ethernet interface. When the client pings the server's PPP IP address, the ping request goes to the PPPoeE-over-VLAN interface and then to the Ethernet interface. The three ping request packets have different packet structures that reflect the differences in the three interfaces.

The figure below also shows a second PPPoE interface, whose parent interface is the Ethernet interface. A PPPoeE interface and a PPPoE-over-VLAN interface can exist simultaneously. The method used to assign IP addresses during the IPCP phase of the PPP call setup in this example means that the peer address for both pppoe1 and pppoe2 interfaces on the client is 10.1.4.1. When the client pings the PPPoE server, the ping request goes out on pppoe1, the PPPoE-over-VLAN interface because that interface was created first. If the PPPoE interface had been created first, the ping request would have gone out that interface instead.

When the client in the figure below puts the second PPPoE interface, **pppoe2**, into service, the server suggests that the client use as its local address the second address in the address pool on the server. The ping requests sent by the server to the client in the figure below shows that **pppoe1** is a PPPoE-over-VLAN interface and that **pppoe2** is a PPPoE interface.

Figure 1. Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN



Parent topic: PPPoE Over VLAN on page 9

6.2. Clients on the Same VLAN and Clients Determine Addresses

You can have a Point-to-Point Protocol over Ethernet (PPOE) over VLAN connection between two clients and a server where the clients determine the PPP IP addresses.

All nodes reside on the same LAN. All nodes have a VLAN interface that has been assigned the same VLAN tag.

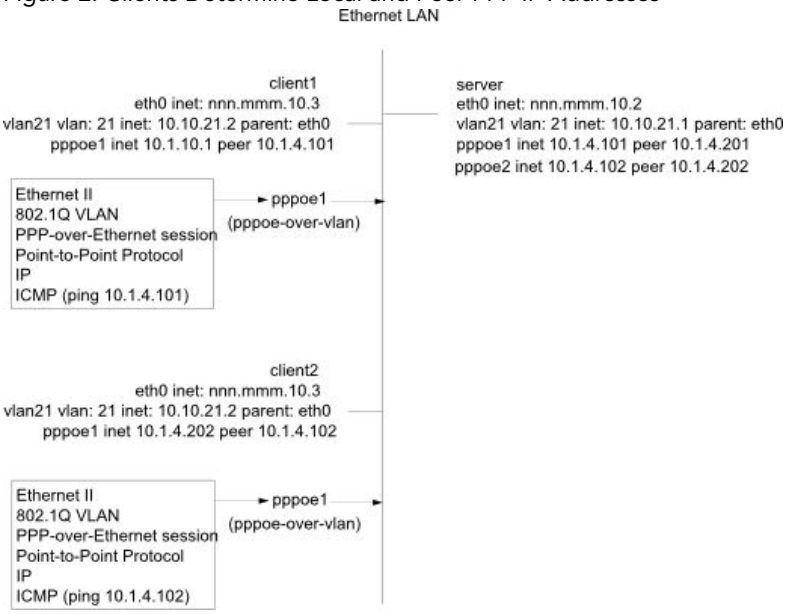
The parent physical interface for the VLAN interface on each node is the physical interface to the LAN. A PPPoE interface is created on the client using a console command and is associated with the VLAN interface using another console command. The PPPoE-over-VLAN interface is put into service using a console command, and this action triggers the sending of a connection request to the server. The exchange of control packets between the client and the server results in a PPPoE interface being established on the server, and this interface uses the VLAN interface, and indirectly the Ethernet interface, for transport.

For an overview of the run-time configuration, see Run-Time Configuration Summary for the PPPoE Server on page 13 and Run-Time Configuration Summary for the Client on page 13.

The IP addresses used by the server and clients on the PPPoE-over-VLAN connection are negotiated during the Internet Protocol Control Protocol (IPCP) phase of the PPP call setup. The IPCP IP addresses shown for the client and the server in the figure below are the result of applying the pattern shown in *Case 4* in the table *Configuring IPCP IP Address Negotiation Between Two Nodes* in PPP

Configuration Parameters on page 18. The local address is set statically on the client, and the server requests an address from the client and the client provides an address stored in the PPP_IPCP_PEER_IPV4_ADDRESS parameter or the value input at the console at run time.

Figure 2. Clients Determine Local and Peer PPP IP Addresses



Parent topic: PPPoE Over VLAN on page 9

6.3. Clients on the Same VLAN and Server Determines Addresses

You can have a Point-to-Point Protocol over Ethernet (PPOE) over VLAN connection between two clients and a server where the server determines the PPP IP addresses.

This is a variant of the example in PPPoE-over-VLAN and PPPoE Connections on the Same Client on page 9. The server in that example is configured to assign clients Internet Protocol Control Protocol (IPCP) addresses from a pool. The client in that example has two PPPoE connections to the server, one over VLAN and one directly over Ethernet. The server assigned the client's local IP address for each connection from a pool.

In this example there are two clients, and each has a PPPoE-over-VLAN connection to the server. The configuration of the clients and the server for the purpose of negotiating the IPCP addresses during the PPP call setup is identical to that of the server and the client with the 2 PPPoE connections.

Parent topic: PPPoE Over VLAN on page 9

6.4. Clients on Different VLANs and the Server Determines Addresses

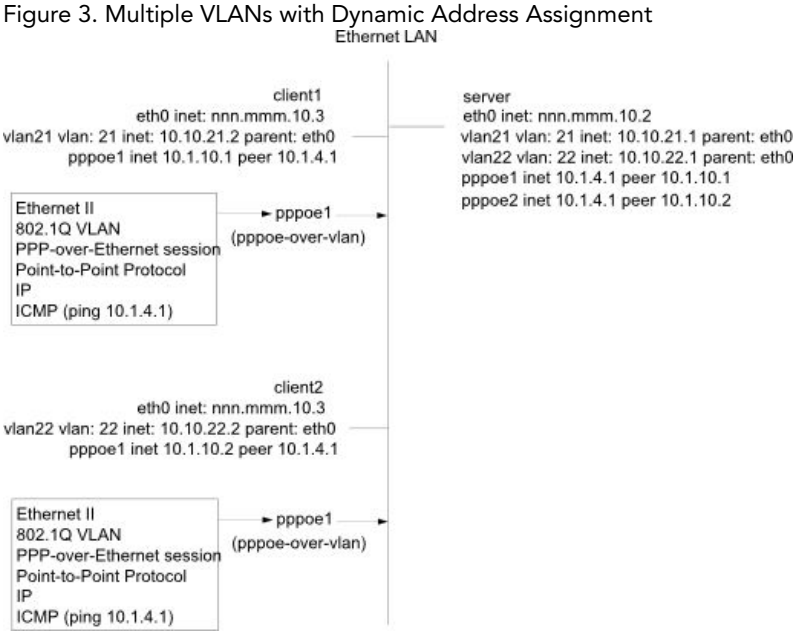
The server can determine IP addresses for clients of different VLANs.

The figure below illustrates the connection of two clients each on a different VLAN to a server. All nodes reside on the same LAN. Each client has a VLAN interface that has the same VLAN tag as a VLAN interface on the server. The server has two VLAN interfaces, one for each client. The parent physical interface for the VLAN interface on each node is the physical interface to the LAN. A Point-to-Point Protocol over Ethernet (PPoE) interface is created on the client using a console command and is associated with the VLAN interface using another console command. The PPPoE-over-VLAN interface is put into service using a console command, and this action triggers the sending of a connection request to the server. The exchange of control packets between the client and the server results in a PPPoE interface being established on the server, and this interface uses the VLAN interface, and indirectly the Ethernet interface, for transport.

For an overview of the run time configuration, see Run-Time Configuration Summary for the PPPoE Server on page 13 and Run-Time Configuration Summary for the Client on page 13.

The IP addresses used by the server and client on the PPPoE-over-VLAN connection and the PPPoE connection are negotiated during the IPCP phase of the PPP call setup. The IPCP IP addresses shown for the client and the server in the figure below are the result of applying the pattern shown in *Case 1* or *Case 2* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table in PPP IPCP Configuration Parameters on page 23. The local address is set statically in the server, and the client requests an address from the server and the server provides an address from an address pool.

Figure 3. Multiple VLANs with Dynamic Address Assignment



Parent topic: PPPoE Over VLAN on page 9

6.5. Build-Time Configuration Summary for the Server and the Client

Configuring and building a VxWorks image that supports the PPPoE over VLAN protocol is the same as doing it for Point-to-Point Protocol over Ethernet (PPPoE).

About This Task

You can use the same VxWorks source build (VSB) project for both the server and the client. You can also use the same VxWorks image project (VIP) for both the server and the client as along as the PPPoE server functionality is enabled at runtime in the server and disabled in the client.

Procedure

1. Create a VSB project and build it.
2. If you plan to configure behavior at runtime, create a single VIP for the server and the client or, if you plan to configure behavior at build time, create a separate VIP for each of them. Include the PPP and PPPoE components to get the PPPoE functionality. Include the pppconfig and sysvar command components, which are used to modify the values of the PPP parameters at runtime. Include the ifconfig command component, which is used to create and manage the VLAN and PPoE interfaces.
3. If the VIP is being used for both the client and the server, leave the PPPoE server functionality disabled in the VIP. If you are using two VIPs, enable the server parameter in the server VIP and leave it disabled in the client VIP.
4. Optionally, configure PPP authentication.

5. Configure the PPP Internet Protocol Control Protocol (IPCP) parameters. This can be done at runtime.
6. Build the VIP.

Parent topic: PPPoE Over VLAN on page 9

6.6. Run-Time Configuration Summary for the PPPoE Server

How to configure the Point-to-Point Protocol over Ethernet (PPPoE) server at runtime.

Procedure

1. Enable the node as a PPPoE server.
2. Create the VLAN pseudo-interface.
3. Associate the VLAN pseudo-interface with an Ethernet interface.
4. Bring the VLAN pseudo-interface into service.
5. Make any necessary modifications to the PPP Internet Protocol Control Protocol (IPCP) parameters.

Parent topic: PPPoE Over VLAN on page 9

6.7. Run-Time Configuration Summary for the Client

How to configure the Point-to-Point Protocol over Ethernet (PPPoE) client at runtime.

Procedure

1. Create the VLAN pseudo-interface.
2. Associate the VLAN pseudo-interface with an Ethernet interface.
3. Bring the VLAN pseudo-interface into service.
4. Make any necessary modifications to the PPP Internet Protocol Control Protocol (IPCP) parameters.
5. Create the PPPoE interface.
6. Associate the PPPoE interface with the VLAN pseudo-interface.
7. Bring the PPPoE interface into service.

Parent topic: PPPoE Over VLAN on page 9

7. Van Jacobson Header Compression

Point-to-Point Protocol (PPP) supports Van Jacobson header compression. Van Jacobson compression (VJC) TCP/IP header compression reduces the size of TCP/IP headers. This can be a significant improvement on slow serial lines, particularly for interactive traffic.

The VJC of TCP/IP headers results in smaller packets. However, it also results in additional processing steps for encoding and lookup operations.

VJC is separate from the Protocol-Field-Compression and the Address-and-Control-Field-Compression PPP options, which control the format of fields in the PPP header. VJC applies to the TCP/IP headers in the packets that PPP encapsulates. VJC is specifically for use over serial lines and is not used for Point-to-Point Protocol over Ethernet (PPPoE) connections, IPv6, or for non-TCP packets.

A VJC configuration option indicates the ability to receive compressed packets. Each end of the link must separately request this option if you want bi-directional compression.

Previous topic: PPPoE Over VLAN on page 9

3. CONFIGURATION AND BUILD

1. PPP Configuration and Build Workflow

The VxWorks 7 installation includes source code but does not include pre-built kernel libraries. Before beginning any VxWorks development, you must build the VxWorks kernel libraries (a VxWorks source build (VSB) project). You must also build the VxWorks run-time kernel image based on the kernel libraries (a VxWorks image project (VIP)).

For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the VxWorks 7 Configuration and Build Guide.

For a list of available source build configuration options for PPP, see PPP Source Build Configuration Options (VSB) on page 15.

For a list of available kernel configuration components for PPP, see PPP Kernel Configuration Components (VIP) on page 17.

Next topic: PPP Source Build Configuration Options (VSB) on page 15

2. PPP Source Build Configuration Options (VSB)

Point-to-Point Protocol (PPP) configuration includes VxWorks source build (VSB) options.

Table 1. PPP VSB Options

Name	Description
IPNET_LINKPROTO	Enable link protocol libraries. This component defines IPNET_PPP.
IPPPP_MP	Enable the multilink protocol for PPP in the VSB.
OPENSSL	Enable cryptography support for the network stack. For more information, see the VxWorks 7 Cryptography Libraries Programmer's Guide.

PPP Definitions in ipppp_config.h

The following macros and parameters are defined in **ipppp_config.h** and must be set before you build a VSB.

IPPPP_USE_AUTH_PAP

Includes support for PAP authentication. Includes IPPPP_USE_AUTH.

IPPPP_USE_AUTH_CHAP

Includes support for CHAP authentication. Includes IPPPP_USE_AUTH.

IPPPP_USE_RFC1877_CLIENT

Includes support for RFC 1877 client support for primary and secondary DNS/NBNS addresses.

IPPPP_USE_RFC1877_SERVER

Includes support for RFC 1877 server support for primary and secondary DNS/NBNS addresses.

IPPPP_USE_PPPOE

Includes support for PPPoE.

IPPPP_USE_PPPMP

Includes support for PPPMP

IPPPP_MAX_LINKS

You must define the following before the VSB library is built:

If multilink is going to be supported, set IPPPP_MAX_LINKS as the maximum size of a bundle.

IPPPP_PPPMP_MRRU_MAX

Default: 3000

The default value of this parameter is related to use of the sysvar run-time parameter pppp.mp.mrru which must be less than or equal to the value defined here. Ordinarily, this default value does not need to be changed.

IPPPP_USE_VJCOMP

Controls whether Van Jacobson Header compression (VJC) is included in PPP at library build time.

After defining IPPPP_USE_VJCOMP and rebuilding the VSB library, the VJC component INCLUDE_IPPPP_VJCOMP is available for configuration in the Workbench 4 kernel configuration. There is one parameter, PPPVJC_ENABLE, that controls whether VJC negotiation is initiated on each new connection. The default is 0, or disabled. To enable it globally, or for a particular interface, use the sysvar command.

For example:

```
# sysvar set -o ipppp.vjc.enable 1
```

IPPPP_VJCOMP_MAX_SLOTS

Default: 16

Controls the maximum number of simultaneous TCP/IP connections. It also controls the size of the structures allocated for use by VJC, and should not be increased arbitrarily. It must not exceed a maximum setting of 255. The default (16) is usually adequate.

IPPPP_VJCOMP_COMP_SLOT_ID

Default: 0

Controls whether the connection ID is sent with every compressed packet. The default is to include it, which adds a difference of one byte to the packet.

IPPPP_PPPOE_USE_MD5

Includes support for the PPPoE cookie (Workbench 4 parameter PPPOE_SECRET_NAME) used for MD5 checksums to thwart denial of service attacks.

Previous topic: PPP Configuration and Build Workflow on page 15
Next topic: PPP Kernel Configuration Components (VIP) on page 17

3. PPP Kernel Configuration Components (VIP)

The required VxWorks image project (VIP) component for Point-to-Point Protocol (PPP) functionality is INCLUDE_IPPPP.

For a list of PPP configuration parameters, see PPP Configuration Parameters on page 18.

Previous topic: PPP Source Build Configuration Options (VSB) on page 15
Next topic: PPP Optional Kernel Configuration Components on page 17

4. PPP Optional Kernel Configuration Components

You can modify the Point-to-Point Protocol (PPP) client behavior with a number of optional kernel configuration components.

Component	Description
INCLUDE_IPPPP_INTERFACE_CONFIG	Include the interface configuration parameters for PPP. See PPP Interface Configuration Parameters on page 26 for a list of PPP interface configuration parameters and their default values
INCLUDE_IPPPP_USERS_CONFIG	Include the user configuration parameters for PPP. See PPP User Configuration Parameters on page 28 for a list of PPP user configuration parameters and their default values
INCLUDE_IPPPPOE	Include the point-to-point protocol over Ethernet. See PPPOE Configuration Parameters on page 29 for a list of PPPOE configuration parameters and their default values.
INCLUDE_IPPPPMP	Include PPP multilink protocol in your VIP. See PPP Multilink Protocol Configuration Parameters on page 31 for a list of PPP MP configuration parameters and their default values.
INCLUDE_IPPPP_VJCOMP	Include PPP TCP/IP header compression from RFC 1144. See PPP Van Jacobson Header Compression Configuration Parameters on page 34 or a list of VJCOMP configuration parameters and their default values.
INCLUDE_IPPPP_CMD	Include the PPP configuration shell command (pppconfig).
INCLUDE_IPIFCONFIG_CMD	Include the interface configuration shell command (ifconfig).
INCLUDE_IPD_CMD	Include the daemon control shell command (ipd).
INCLUDE_IPCOM_SYSLOGD_CMD	Include the system logging shell command (syslog).
INCLUDE_IPCOM_SYSVAR_CMD	Include the system variables shell command (sysvar).

Component	Description
INCLUDE_IPCOM_USE_KEY_DB	Include support for the secure key database to store keys in an encrypted format. The IPCOM_KEY_DB_KEY_ENCRYPTING_KEY parameter must be explicitly set. You must also enable the SEC_CRYPTO VSB option to see this component. For more information, see VxWorks 7 Cryptography Libraries Programmer's Guide.
INCLUDE_IPCOM_KEY_DB_CMD	Include support for the secure key database shell commands to manipulate the key database. You must enable the SEC_CRYPTO VSB option to see this component. For more information, see VxWorks 7 Cryptography Libraries Programmer's Guide.

Previous topic: PPP Kernel Configuration Components (VIP) on page 17
Next topic: PPP Configuration Parameters on page 18

5. PPP Configuration Parameters

You can modify the Point-to-Point Protocol (PPP) client behavior with a number of configuration parameters and system variables.

The table below lists the PPP configuration parameters and system variables (INCLUDE_IPPPP).

Parameter Name, System Variable Name, and Description	Default and Type
PPP_L2TP_LNS_ADDRESS_DEFAULT ipppp.l2tp.lnsAddrDefault Configures a default IP address for PPP over L2TPv2 for a network server. This address will be used by PPP when establishing a PPP over L2TPv2 connection, when no address has been assigned.	**""** char *
PPP_LCP_ECHO_REQ_FAILURE \ ipppp.lcp.echo_failure Sets the maximum number of echo request failures before link termination.	**5** char *
PPP_DEFAULT_BAUDRATE ipppp.baudrate The RS232 serial driver initial baud rate.	**9600** char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_INSTALL_CALLBACK_HOOK When **TRUE** compiles in support for user-defined PPP action callback hooks. See the PPP_ACTION_CALLBACK_HOOK parameter and 5.1 Callback Function, p. 51 for details.	**FALSE** BOOL
PPP_LCP_ECHO_REQ_INTERVAL ipppp.lcp.echo_interval Sets the link control protocol (LCP) echo request output interval in seconds. Set to **0** to disable.	**60** char *
PPP_LCP_MRU ipppp.lcp.mru The LCP maximum receive unit.	**1500** char *
PPP_LCP_MTU ipppp.lcp.mtu The LCP maximum transmit unit (MTU).	**1500** char *
PPP_RUNMODE ipppp.runmode Sets the runmode on a PPPoE interface. The parameter value is an ASCII string with one or more runmode options. There are four runmode options defined: **start** When set, the interface automatically starts when attached. Otherwise requires an IP_SIOCSIFFLAGS ioctl( ) request with the IP_IFF_UP flag. PPPoE interfaces are always started automatically when they are created. **passive** Enables the passive option in the LCP. When set, a PPP interface attempts only once to initiate a connection when started. If no reply is received from the peer after a number of tries, the interface waits passively for a valid LCP configuration request packet from the peer. **exit** Exit LCP after a connection; that is, do not accept new connections unless the PPP interface is reopened with an IP_SIOCSIFFLAGS ioctl( ) request with the IP_IFF_UP flag. **wincompat** Supports connection to a Windows machine using PPP and a null-modem connection. This define is necessary because Windows requires CLIENT/CLIENTSERVER strings before starting standard PPP.	**start, passive, wincompat** char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_AUTH_MODES ipppp.auth Sets the authentication requirements on a PPP interface. The parameter value is an ASCII string with one or more authentication options. There are six options defined. auth This requires the peer to authenticate itself before allowing sending or receiving of IPCP or IPV6CP. noauth This does not require the peer to authenticate itself (that is, do not ask for authentication.) pap The peer must authenticate using the PAP protocol. refuse-pap Do not agree to authenticate using PAP. chap The peer must authenticate using the CHAP protocol. refuse-chap Do not agree to authenticate using CHAP.	**noauth** char *
PPP_FLAGS ipppp.flags Sets behaviors and options on an interface. The parameter value is an ASCII string with one or more flag options. There are two flag options defined: defaultroute This adds a default route to the system routing tables using the peer as the gateway, when the IPCP/IPV6CP negotiation is successfully completed. **proxyarp** This adds a proxy ARP route flag (RTF_PROTO2) to the peer host route when IPCP negotiation is successfully completed. When this is set, the target will act as proxy for the host. One or more additional interfaces on the target must have proxy ARP enabled. ARP requests for the host address that come in on these interfaces will be processed by the target. See the VxWorks 7 Network Stack Programmer's Guidefor information about enabling proxy ARP.	**proxyarp** char *
PPP_PASSWD ipppp.password The PPP client password used when the peer PPP server requires authentication using PAP or CHAP.	**kallekula** char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_USERNAME ipppp.username The PPP client username used when the peer PPP server requires authentication using PAP or CHAP.	**ppp** char *
PPP_IPCP_IPV4_ADDRESS ipppp.ipcp.addr Local IPv4 address. Setting it to **0.0.0.0** means that the stack will ask the peer to suggest its IPv4 address. This suggestion is contained in a PPP IPCP Configuration Request packet. Setting it to a value other than **0.0.0.0** means that the node will use that value as the source IP address for data sent over the PPP connection.	**0.0.0.0** char *
PPP_IPCP_PEER_IPV4_ADDRESS ipppp.ipcp.dstaddr Suggests the peer IPv4 address. The peer IPv4 address is suggested to the peer only if the peer requests it by specifying IP address **0.0.0.0** as a configuration option in a IPCP configuration request. Unexpected behavior can occur if two peers are configured to request an IP address from the other peer and both use the same IP address in response to the request.	**10.1.4.1** char *
PPP_IPCP_PEER_IPV4_POOL_NAME ipppp.ipcp.pool Names an IPv4 address pool used to supply peers with IPv4 addresses. This name is an argument to the pppconfig setpool command used to modify the address range. The range of IPv4 addresses in the pool is defined at build time using the **PPP_IPCP_PEER_IPV4_ADDRESS_POOL** parameter.	**default** char *
PPP_IPCP_PEER_IPV4_ADDRESS_POOL ipppp.ipcp.pool.default Defines the range of IPv4 addresses used to supply peers that request it, with an address. The pool name is defined by the PPP_IPCP_PEER_IPV4_POOL_NAME parameter. The pool is available to servers such as the PPPoE server but not available to clients.	**10.1.3.1-10.1.3.255** char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_IPCP_PRIMARY_DNS_ADDRESS ipppp.ipcp.primary_dns_address PPP client IPv4 primary DNS address. The configured IPv4 address is given to the peer (the client) if the peer requests a primary DNS address.	"" char *
PPP_IPCP_PRIMARY_NBNS_ADDRESS ipppp.ipcp.primary_nbns_address PPP client IPv4 primary NBNS address. The configured IPv4 address is given to the peer if the peer requests a primary NBNS (WINS) address.	"" char *
PPP_IPCP_SECONDARY_DNS_ADDRESS ipppp.ipcp.secondary_dns_address PPP client IPv4 secondary DNS address. The configured IPv4 address is given to the peer if the peer requests a secondary DNS address.	"" char *
PPP_IPCP_SECONDARY_NBNS_ADDRESS ipppp.ipcp.secondary_nbns_address PPP client IPv4 secondary NBNS address. The configured IPv4 address is given to the peer if the peer requests a secondary NBNS (WINS) address.	"" char *
PPP_ACTION_CALLBACK_HOOK Name for callback function. PPP calls this function to handle various PPP actions, such as initialization, startup, peer IPv4 address requests, peer authentication, login, and so on. See the PPP_INSTALL_CALLBACK_HOOK parameter and 5.1 Callback Function, p. 51 for details. PPP_ACTION_CALLBACK_HOOK is applicable only if PPP_INSTALL_CALLBACK_HOOK is set to **TRUE**. To avoid compiler errors, PPP_ACTION_CALLBACK_HOOK is defined as **NULL** by default.	**NULL** funcptr

Configuring Global (PPP), Per-interface, and Per-user Parameters on page 23 Many Point-to-Point Protocol (PPP) configuration parameters can be set on a global, per-interface, or per-user basis. Global parameters apply to all users and interfaces. The per-interface and per-user parameters override the global parameters for a given interface or user.

Previous topic: PPP Optional Kernel Configuration Components on page 17
Next topic: PPP Interface Configuration Parameters on page 26

5.1. Configuring Global (PPP), Per-interface, and Per-user Parameters

Many Point-to-Point Protocol (PPP) configuration parameters can be set on a global, per-interface, or per-user basis. Global parameters apply to all users and interfaces. The per-interface and per-user parameters override the global parameters for a given interface or user.

For example, the authentication mode can be set for all interfaces using the **Authentication mode** (PPP_AUTH_MODES) parameter of the **PPP** (INCLUDE_IPPPP) component. To set an authentication mode for a single interface, use the **Authentication mode** (PPP_IF_AUTH_MODES_LIST) parameter of the **Per-interface configurations** (INCLUDE_IPPPP_INTERFACE_CONFIG) component.

Parameter names that include **LIST**, such as PPP_IF_AUTH_MODES_LIST, accept a list of interfaces or users and corresponding values. Each entry in the list is separated by semicolons. For example, to set the authentication mode for interfaces **ppp0** and **ppp1**, set the PPP_IF_AUTH_MODES_LIST parameter to:

```
"ppp0=pap;ppp1=pap"
```

The per-user parameters are in the **Per-user configurations** (INCLUDE_IPPPP_USERS_CONFIG) component. Most per-interface parameters are in the **Per-interface configurations** (INCLUDE_IPPPP_INTERFACE_CONFIG) component. The **PPPOE** (INCLUDE_IPPPPOE) component does have the following per-interface parameters:

- PPPOE_IF_MAX_ETH_SESSIONS_LIST
- PPPOE_IF_SERVICE_NAME_LIST

PPP IPCP Configuration Parameters on page 23 The PPP_IPCP_* parameters of the PPP component govern the behavior of the Internet Protocol Control Protocol (IPCP) phase of the Point-to-Point Protocol (PPP) connection setup.

Parent topic: PPP Configuration Parameters on page 18

PPP IPCP Configuration Parameters

The PPP_IPCP_* parameters of the PPP component govern the behavior of the Internet Protocol Control Protocol (IPCP) phase of the Point-to-Point Protocol (PPP) connection setup.

Section 3.3 of RFC 1332 describes the IP-Address configuration option used to negotiate the IP addresses used by the two ends of the connection.

The INCLUDE_IPPPP parameters used to control the IP address negotiation are PPP_IPCP_IPV4_ADDRESS, PPP_IPCP_PEER_IPV4_ADDRESS, and PPP_IPCP_PEER_IPV4_ADDRESS_POOL.

In the cases where one peer uses the Wind River implementation of the protocol and the other peer does not, use these parameters to configure the local and peer addresses of a PPP connection in the following ways:

- the Wind River peer does not solicit an address from the other node and uses the non-pool address to respond to requests from the other node for an address (*Case 1* in the *Configuring IPCP Address Negotiation* table.)
- the Wind River peer does not solicit an address from the other node and uses a pool address to respond to requests from the other node for an address (*Case 2* in the *Configuring IPCP Address Negotiation* table.
- the Wind River peer solicits an address from the other node and uses the non-pool address to respond to requests from the other node for an address (*Case 3* in the *Configuring IPCP Address Negotiation* table.
- the Wind River peer solicits an address from the other node and uses a pool address to respond to requests from the other node for an address (*Case 4* in the *Configuring IPCP Address Negotiation* table.

The *Configuring IPCP Address Negotiation* table in illustrates these parameter configuration cases. See Setting the PPP System Variables on page 41 for the runtime commands used to configure these cases.

Note: In the *Configuring IPCP Address Negotiation* and *Configuring IPCP IP Address Negotiation Between Two Nodes* tables placeholders are used to signify values. The values of *local-ipv4-address*, *peer-ipv4-address* and *ipv4-address-range* must be set so that when the negotiation rules are applied the two peers end up with different addresses. *ipv4-address-range* denotes a series of addresses that have no gaps. **0.0.0.0** is the invalid address. **unset** signifies that the parameter value has not been configured or has been made unavailable using a runtime command. **do not care** signifies that the parameter value plays no part in the behavior of the case.

Configuring IPCP Address Negotiation

Case	Parameter	Setting
Case 1: The Wind River peer does not solicit an address from the other node and uses the non-pool address to respond to requests from the other node for an address.	PPP_IPCP_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	*local-ipv4-address* *peer-ipv4-address* **do not care**
Case 2: The Wind River peer does not solicit an address from the other node and uses a pool address to respond to requests from the other node for an address.	PPP_IPCP_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	*local-ipv4-address* **0.0.0.0** or **unset** *ipv4-address-range*
Case 3: The Wind River peer solicits an address from the other node and uses the non-pool address to respond to requests from the other node for an address.	PPP_IPCP_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	**0.0.0.0** or **unset** *peer-ipv4-address* **do not care**
Case 4: The Wind River peer solicits an address from the other node and uses a pool address to respond to requests from the other node for an address.	**PPP_IPCP_IPV4_ADDRESS** PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	**0.0.0.0** or **unset** **0.0.0.0** or **unset** *ipv4-address-range*

In the cases where the both peers use the Wind River implementation of the protocol, you can use these parameters to configure the local and peer addresses of a PPP connection in the following ways:

- one node uses an address that is configured locally and the other node solicits an address from the first node and get a pool address (*Case 1* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table.
- both nodes use addresses that are configured locally (*Case 2* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table.)
- one node uses an address that is configured locally, and the other node solicits an address from the first node and gets the non-pool address (*Case 3* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table.)
- one node solicits an address from the other node and gets the non-pool address; the second node solicits an address from the first node and gets a pool address (*Case 4* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table.)
- one node solicits an address from the other node and gets a pool address; the second node solicits an address from the first node and gets a pool address (*Case 5* in the *Configuring IPCP IP Address Negotiation Between Two Nodes* table.)

The *Configuring IPCP IP Address Negotiation Between Two Nodes* table illustrates the parameter configuration cases applicable to connections between two peers that use the Wind River implementation.

Note: In the *Configuring IPCP IP Address Negotiation Between Two Nodes* table *ipv4-address-range* denotes a series of addresses that have no gaps. **0.0.0.0** is the invalid address. **unset** signifies that the parameter value has not been configured or has been made unavailable using a runtime command. **do not care** signifies that the parameter value plays no part in the behavior of the case.

Configuring IPCP IP Address Negotiation Between Two Nodes

Case	Parameter	Wind River Node1	Wind River Node2
Case 1: One node uses an address that is configured locally and the other node solicits an address from the first node and get a pool address.	PPP_IPCP_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	*local-ipv4-address* **0.0.0.0** or **unset** *ipv4-address-range*	**0.0.0.0** or **unset** **do not care** **do not care**
Case 2: Both nodes use addresses that are configured locally.	**PPP_IPCP_IPV4_ADDRESS** PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	*local-ipv4-address* **do not care** **do not care**	*local-ipv4-address* **do not care** **do not care**
Case 3: One node uses an address that is configured locally, and the other node solicits an address from the first node and gets the non-pool address.	**PPP_IPCP_IPV4_ADDRESS** PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	**0.0.0.0** or **unset** **do not care** **do not care**	*local-ipv4-address* *peer-ipv4-address* **do not care**
Case 4: One node solicits an address from the other node and gets the non-pool address; the second node solicits an address from the first node and gets a pool address.	PPP_IPCP_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	**0.0.0.0** or **unset** *peer-ipv4-address* **do not care**	**0.0.0.0** or **unset** **0.0.0.0** or **unset** *ipv4-address-range*
Case 5: One node solicits an address from the other node and gets a pool address; the second node solicits an address from the first node and gets a pool address.	**PPP_IPCP_IPV4_ADDRESS** PPP_IPCP_PEER_IPV4_ADDRESS PPP_IPCP_PEER_IPV4_ADDRESS_POOL	**0.0.0.0** or **unset** **0.0.0.0** or **unset** *ipv4-address-range*	**0.0.0.0** or **unset** **0.0.0.0** or **unset** *ipv4-address-range*

Requesting an IP Address from the Peer

During the IPCP phase of the PPP connection setup the peers exchange IPCP configuration requests to negotiate connection parameters. Among these parameters are the local and peer IP addresses used for the data packets exchanged during the call. A node sends a configuration request to its peer containing the IP address it will use as its local address during the call. The value sent in this request is the value of the PPP_IPCP_IPV4_ADDRESS parameter. During VIP creation the PPP_IPCP_IPV4_ADDRESS parameter is initialized to **0.0.0.0**, the invalid IP address.

When Wind River peer receives a configuration request with the invalid IP address, it checks the PPP_IPCP_PEER_IPV4_ADDRESS parameter for a valid IP address. If that parameter contains a valid address, the Wind River peer sends a configuration NAK containing that address to the requestor.

During VIP creation the PPP_IPCP_PEER_IPV4_ADDRESS parameter is initialized with a valid IP address. If it has been assigned an invalid IP address (**0.0.0.0**) at build time or runtime or has been unset at run time, the peer checks for the existence of the **PPP_IPCP_PEEP_IPV4_ADDRESS_POOL** parameter. At VIP creation the address pool is initialized with a valid range of IP addresses. The Wind River peer takes the next available address from this pool and sends it to the requestor.

When the Wind River requestor receives the NAK, it signals its acceptance of the address by returning it to the peer in a configuration request. A Wind River requestor accepts any valid IP address that is not a multicast or broadcast address. When the peer receives the configuration request containing the IP address that it originally suggested, it acknowledges the request.

If the address sent by the peer in the NAK is a multicast or broadcast address, the requestor sends a config reject to the peer. The requestor then waits for a config request timer to expire. When the timeout occurs, the requestor restarts the IPCP process over again unless the maximum number of config request timeouts have occurred. See RFC 1332 for more information on IPCP. IPCP uses the same state machine as LCP. See RFC 1661 for information about this state machine.

Note:

According to RFC 1332, section 3.3, the suggested IP address in a NAK must be an address that the node sending the NAK would approve of. So it is unlikely that a Wind River requestor would ever find an unacceptable IP address in the NAK and would ever have occasion to send a configuration reject message to the peer.

Parent topic: Configuring Global (PPP), Per-interface, and Per-user Parameters on page 23

6. PPP Interface Configuration Parameters

List of Point-to-Point Protocol (PPP) interface configuration parameters and system variables (INCLUDE_IPPPP_INTERFACE_CONFIG).

Parameter Name, System Variable Name, and Description	Default and Type
PPP_IF_AUTH_MODES_LIST ipcom.if. *ifname* **.ipppp.auth** Authentication mode.	"" char *
PPP_IF_LCP_ECHO_REQ_FAILURE_LIST ipcom.if. *ifname* .ipppp.lcp.echo_failure Echo failure.	"" char *
PPP_IF_IPCP_PEER_IPV4_ADDRESS_POOL_LIST **ipcom.if.** *ifname* .ipppp.ipcp.pool IPv4 pool address list.	"" char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_IF_DEFAULT_BAUDRATE_LIST ipcom.if.*ifname* .ipppp.baudrate Initial baud rate.	"" char *
PPP_IF_LCP_ECHO_REQ_INTERVAL_LIST ipcom.if.*ifname* .ipppp.lcp.echo_interval Interface LCP echo request interval.	"" char *
PPP_IF_RUNMODE_LIST ipcom.if.*ifname* .ipppp.runmode Interface runmode.	"" char *
PPP_IF_LCP_MRU_LIST ipcom.if.*ifname* .ipppp.lcp.mru LCP maximum receive unit.	"" char *
PPP_IF_LCP_MTU_LIST ipcom.if.*ifname* .ipppp.lcp.mtu LCP maximum transmit unit.	"" char *
PPP_IF_FLAGS_LIST ipcom.if.*ifname* .ipppp.flags PPP flags.	"" char *
PPP_IF_IPCP_PEER_IPV4_ADDRESS_LIST ipcom.if.*ifname* .ipppp.ipcp.dstaddr Peer IPv4 address list.	"" char *
PPP_IF_IPCP_PRIMARY_DNS_ADDRESS_LIST ipcom.if.*ifname* .ipppp.ipcp.primary_dns_address Primary DNS address list.	"" char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_IF_IPCP_SECONDARY_DNS_ADDRESS_LIST ipcom.if.*ifname* .ipppp.ipcp.secondary_dns_address Secondary DNS address list.	"" char *
PPP_IF_IPCP_IP4V_ADDRESS_LIST ipcom.if.*ifname* .ipppp.ipcp.addr Local IPv4 address list.	"" char *

Previous topic: PPP Configuration Parameters on page 18
Next topic: PPP User Configuration Parameters on page 28

7. PPP User Configuration Parameters

List of Point-to-Point Protocol (PPP) user configuration parameters and system variables (INCLUDE_IPPPP_USERS_CONFIG).

Parameter Name, System Variable Name, and Description	Default and Type
PPP_USERS_IPCP_PEER_IPV4_ADDRESS_POOL_LIST ipcom.users.*user* .ipppp.ipcp.pool Users IPv4 peer address pool list.	"" char *
PPP_USERS_LCP_ECHO_REQ_INTERVAL_LIST ipcom.users.*user* **.ipppp.lcp.echo_interval** Users LCP echo request interval.	"" char *
PPP_USERS_IPCP_PEER_IPV4_ADDRESS_LIST ipcom.users.*user* .ipppp.ipcp.dstaddr Users peer IPv4 address list.	"" char *
PPP_USERS_IPCP_IPV4_ADDRESS_LIST ipcom.users.*user* .ipppp.ipcp.addr Users local IPv4 address list.	"" char *

Parameter Name, System Variable Name, and Description	Default and Type
PPP_USERS_IPCP_PRIMARY_DNS_ADDRESS_LIST ipcom.users.*user* .ipppp.ipcp.primary_dns_address Users primary DNS address list.	"" char *
PPP_USERS_IPCP_SECONDARY_DNS_ADDRESS_LIST ipcom.users.*user* .ipppp.ipcp.secondary_dns_address Users secondary DNS address list.	"" char *

Previous topic: PPP Interface Configuration Parameters on page 26
Next topic: PPPOE Configuration Parameters on page 29

8. PPPOE Configuration Parameters

List of Point-to-Point Protocol over Ethernet (PPPOE) configuration parameters and system variables (INCLUDE_IPPPPOE).

Parameter Name, System Variable Name, and Description	Default and Type
PPPOE_AC_NAME ipppp.pppoe.ac_name The PPPoE access concentrator name (AC-Name).	**Windriver PPPoE Server** char *
PPPOE_SERVER ipppp.pppoe.server Enables a PPPoE server on a device or an interface. If the option is set to "1", the PPPoE server is enabled. If the option is set to "0", the PPPoE server is disabled for the whole device or for a specific interface.	**0** char *
PPPOE_SECRET_NAME ipppp.pppoe.secret_name Secret name for a PPPoE cookie. Use this parameter in an MD5 checksum calculation to thwart denial-of-service (DoS) attacks. The definition of the string does not need to match any remote configuration.	**puttoanythinghere** char *

Parameter Name, System Variable Name, and Description	Default and Type
Wind River recommends that you change this value from its default setting.	
PPPOE_MAX_SESSIONS ipppp.pppoe.max_sessions Maximum number of PPPoE sessions allowed before incoming PPPoE PADI packets are ignored. That is, the PPPoE server will not accept any more connections.	**16** char *
PPPOE_MAX_ETH_SESSIONS ipppp.pppoe.max_eth_sessions Global setting for the maximum number of PPPoE sessions per Ethernet interface.	**8** char *
PPPOE_IF_MAX_ETH_SESSIONS_LIST ipcom.if. *ifname* .ipppp.pppoe.max_eth_sessions Maximum number of PPPoE sessions per Ethernet interface, if the maximum number varies by interface. Note that the interface name used is the Ethernet interface name and not the PPPoE interface name. This parameter cannot be configured from the command line.	**""** char *
PPPOE_SERVICE_NAME ipppp.pppoe.service_name This is the service name the stack accepts. Do not set this option to accept the **"any"** service name.	**myservice** char *
PPPOE_IF_SERVICE_NAME_LIST ipcom.if. *ifname* .ipppp.pppoe.service_name Per-interface list of service names that the stack accepts. Do not set this option to accept the **"any"** service name.	**IP_NULL** char *

Configuring a Server and Client for PPPoE over VLAN Connections on page 31 This topic documents the VSB components and VIP components and parameters required to implement PPPoE over VLAN connections.

Previous topic: PPP User Configuration Parameters on page 28

Next topic: PPP Multilink Protocol Configuration Parameters on page 31

8.1. Configuring a Server and Client for PPPoE over VLAN Connections

This topic documents the VSB components and VIP components and parameters required to implement PPPoE over VLAN connections.

VxWorks Source Build (VSB)

VLAN and Point-to-Point Protocol (PPP) support is enabled by default in the VxWorks Source Build (VSB). Just create and build a VSB.

VxWorks Image Project (VIP)

VLAN support is included by default in the VIP. PPP support must be included explicitly. Create a VIP for the client and one for the server if you plan to configure behavior at build time by setting parameters. Create one VIP to be used for both the client and the server if you plan to configure behavior at run time. Do the following:

Procedure

1. Create the VIP.
2. Include the INCLUDE_IPPPP component. The INCLUDE_IPPPPOE component is included automatically as a dependency.
3. Include the INCLUDE_IPPPP_CMD. component to get the pppconfig command, which is used to modify and monitor configuration values at run time.
4. Include the INCLUDE_IPCOM_SYSVAR_CMD. component to get the sysvar command, which is used to modify and monitor configuration values at run time.
5. Include the INCLUDE_IPIFCONFIG_CMD. component; this contains the ifconfig command which is used to create and manage the VLAN and PPoE interfaces.
6. Optionally configure PPP authentication. The default is no authentication.
7. Optionally configure the IPCP parameters. See PPP IPCP Configuration Parameters on page 23.
8. Build the VIP.

Parent topic: PPPOE Configuration Parameters on page 29

9. PPP Multilink Protocol Configuration Parameters

List of the Point-to-Point Protocol (PPP) Multilink Protocol (MP) configuration parameters and system variables (INCLUDE_IPPPPMP).

Parameter Name, System Variable Name, and Description	Default and Type
PPPMP_ENABLE ipppp.mp.enable Enables PPP MP. 1=enabled, 0=disabled.	**0** char *
PPPMP_FRAG_DISABLE ipppp.mp.frag_disable Controls the packet fragmentation policy used by multilink, as discussed in RFC 1990, section 4.1. The default behavior is to fragment an outgoing packet among the available links. When fragmentation is disabled, and a link is available with sufficient MTU, the packet is sent unfragmented. To disable fragmentation, set this parameter to 1. The default value is 0.	**0** char *
PPPMP_MRRU ipppp.mp.mrru Sets the Maximum Received Reconstructed Unit.	**1500** char *
PPPMP_PPPOE_LINKS ipppp.pppoe.mp_links Number of links which will be started for multilink use when a PPPoE interface is brought up (PPPoE requires the component INCLUDE_IPPPOE).	**1** char *
PPPMP_SSN ipppp.mp.ssn Controls Short Sequence numbers. In PPP multilink, packets are transferred as multilink fragments, each one having a sequence number to aid in reconstruction. By default sequence numbers are 24 bits, but 16-bits can be negotiated to save packet overhead. The PPPMP_SSN parameter enables this negotiation, which is described in RFC 1990. To enable short sequence negotiation, set this parameter to 1. The default value is 0.	**0** char *

Configuring and Building PPP MP on page 33 Point-to-Point Protocol (PPP) Multilink Protocol (MP) is an optional feature that is not included in the binary files. To enable the feature create and rebuild a new VSB.
PPP Multilink Connections Configuration on page 33 Point-to-Point Protocol (PPP) Multilink is enabled by the PPPMP_ENABLE parameter or by the system variable *ipppp.mp.enable* at runtime. When enabled, a multilink connection can be created manually using the pppconfig command, or automatically upon receipt of a configuration request (which includes the Multilink MRRU LCP option) on a Multilink-enabled interface.

Previous topic: PPPOE Configuration Parameters on page 29
Next topic: PPP Van Jacobson Header Compression Configuration Parameters on page 34

9.1. Configuring and Building PPP MP

Point-to-Point Protocol (PPP) Multilink Protocol (MP) is an optional feature that is not included in the binary files. To enable the feature create and rebuild a new VSB.

About This Task

Point-to-Point Protocol (PPP) Multilink Protocol (MP) and Van Jacobson Compression (VJC) are not compatible, since each sets the PPP packet protocol. If PPP MP is enabled, PPP will attempt to negotiate it and if it succeeds, then VJC is not possible on that link. PPP MP takes precedence because it is negotiated by Link Control Protocol (LCP), whereas VJC negotiation occurs in Internet Protocol Control Protocol (IPCP).

The PPP binary files include the required PPP components only. Optional features, such as PPP MP and VJC, are not included in the binary files. If you want to enable optional features, you must configure PPP appropriately and create a new VSB project and rebuild it. The following steps describe how to enable PPP MP.

Procedure

1. Edit the configuration file **ipppp_config.h** to define the maximum bundle size, IPPPP_MAX_LINKS.

   Edit the IPPPP_MAX_LINKS setting. IPPPP_MAX_LINKS is a build parameter for the VSB library build and cannot be changed in Workbench 4. The value can be set to any appropriate number, with the consideration that memory is consumed proportionately. IPPPP_MAX_LINKS should be set large enough to accommodate the largest expected bundle, but it should not be any larger than necessary due to the memory impact. There is a memory penalty, modulo the size of IPPPP_MAX_LINKS, for enabling multilink for every PPP or PPPoE interface. Note that IPPPP_MAX_LINKS should always be set to 1 when PPP MP is not enabled.

2. Configure and build the VSB after enabling the IPPPP_MP option and setting IPPPP_MAX_LINKS in **ipppp_config.h**.
3. Next, configure a VIP project that includes the VSB. You must include these required PPP components in the VIP project: INCLUDE_IPPPP and INCLUDE_IPPPP_CMD.
4. Configure the following parameters:
   - PPPMP_ENABLE - This must be set to 1 to enable PPP Multilink.
   - PPPMP_MRRU - The default is 1500.
   - PPPMP_PPPOE_LINKS - This sets the number of links created on a PPPoE interface when it is brought up. The default is 1; it must be less than IPPPP_MAX_LINKS.
5. Build the VIP project. You can also use the wrtool command-line utility to configure and build the VSB and VIP projects.

Parent topic: PPP Multilink Protocol Configuration Parameters on page 31

9.2. PPP Multilink Connections Configuration

Point-to-Point Protocol (PPP) Multilink is enabled by the PPPMP_ENABLE parameter or by the system variable *ipppp.mp.enable* at runtime. When enabled, a multilink connection can be created manually using the pppconfig command, or automatically upon receipt of a configuration request (which includes the Multilink MRRU LCP option) on a Multilink-enabled interface.

A PPP Multilink bundle is created when all of the following are true:

- The component INCLUDE_IPPPPMP is included in the build.
- The *ipppp.mp.enable* **sysvar** is enabled.
- LCP link configuration negotiation concludes with the PPP Multilink MRRU variable included.
- A bundle does not already exist with that peer.

Existing bundles are checked and tested for matching endpoint discriminators (if not null) and authentication.

All links in a bundle must have the same endpoint discriminator value, because they all have to be with the same peer. A single interface cannot have links with more than one peer; multiple links must all be in a bundle.

When a link comes up, PPP checks to see if a bundle already exists with that peer (as indicated by the endpoint discriminator value) and, if not, it starts a new bundle.

In the case of a PPPoE interface with multiple links (PPPMP_PPPOE_LINKS > 1), if a subsequent link's endpoint discriminator value does not match the bundle endpoint discriminator value, the link will be dropped.

Parent topic: PPP Multilink Protocol Configuration Parameters on page 31

10. PPP Van Jacobson Header Compression Configuration Parameters

Defines the Point-to-Point Protocol (PPP) VJCOMP configuration parameters and system variables (INCLUDE_IPPPP_VJCOMP).

Parameter Name, System Variable Name, and Description	Default and Type
PPPVJC_ENABLE *ipppp.vjc.enable* Controls if VJC negotiation is initiated on each new connection. The default is 0 (disabled). Set to 1 to enable.	**0** char *

PPP MP and VJC are not compatible, since each sets the PPP packet protocol. If PPP MP is enabled, PPP will attempt to negotiate it and if it succeeds, then VJC is not possible on that link. PPP MP takes precedence because it is negotiated by LCP, whereas VJC negotiation occurs in IPCP.

PPP Configuring Van Jacobson Header Compression on page 34 Use this procedure to configure and build a VxWorks image that supports Van Jacobson Header Compression (VJC).

Previous topic: PPP Multilink Protocol Configuration Parameters on page 31
Next topic: PPPConfiguring an Additional Interface at Build Time on page 35

10.1. PPP Configuring Van Jacobson Header Compression

Use this procedure to configure and build a VxWorks image that supports Van Jacobson Header Compression (VJC).

WNDRVR

About This Task

Procedure

1. In **ipppp_config.h** uncomment the following lines:

   #ifdef IPCOM_USE_INET

   #define IPPPP_USE_VJCOMP

   #endif

2. Create and build the VxWorks Source Build (VSB) project.
3. Create a VxWorks Image Project (VIP) and include the INCLUDE_IPPPP_VJCOMP component.
4. Set the parameter PPPVJC_ENABLE to 1. See the table in PPP Van Jacobson Header Compression Configuration Parameters on page 34.
5. Build the VIP.

Parent topic: PPP Van Jacobson Header Compression Configuration Parameters on page 34

11. PPPConfiguring an Additional Interface at Build Time

To configure an interface at build time, include an INCLUDE_IPNET_IFCONFIG_*N* component (one for each interface). Each of these components contains an IFCONFIG_*N* parameter.

For each IFCONFIG_*N*, edit the following fields:

ifname

Specifies the name of the Ethernet interface, for example, ifname fei0. If the interface name is missing after ifname (the default setting), the END device name will be used.

devname

Specifies the driver to which this interface should attach itself, for example, fei0. The default setting driver instructs VxWorks to retrieve the device name from the device boot parameters.

inet

Specifies the interface IPv4 address and subnet, for example, inet 10.1.2.100/24. Instead of IPv4 address, the following commands can also be used:

inet driver (default)

Specifies that the address and mask should be read from the BSP.

inet dhcp

Specifies that the address and mask should be received from a Dynamic Host Control Protocol (DHCP) server. The gateway might also be received from that server (depending on the DHCP server configuration).

inet rarp

Specifies that the address and mask should be received from an Reverse Address Resolution Protocol (RARP) server.

WNDRVR

gateway

Specifies the default gateway used for IPv4, for example, gateway 10.1.2.1. Only one default gateway can be specified. gateway driver can be used to take the gateway from the boot parameters.

inet6

Specifies the interface IPv6 address and subnet, for example, inet6 3ffe:1:2:3::4/64. The tentative keyword can be inserted before the address if the stack should perform duplicate address detection on the address before assigning it to the interface, for example, tentative 3ffe:1:2:3::4/64.

gateway6

The default gateway used for IPv6. Only one default gateway can be specified.

To configure an interface at run time, see PPP Configuring an Additional Interface at Run Time on page 42.

Previous topic: PPP Van Jacobson Header Compression Configuration Parameters on page 34

4. SHELL COMMANDS

1. About PPP Shell Commands

Several useful shell commands for configuring and interrogating the Point-to-Point Protocol (PPP) are available.

- You can use the pppconfig shell command to configure PPP.
- You can use the sysvar shell command to modify PPP configuration parameters.
- You can use ioctl( ) requests to change driver or link layer parameters.
- You can use the ifconfig shell command to enable or disable PPP on an interface and to display the current status of an interface.
- You can use the ipd shell command to view the state of the PPP process.
- You can use the keydb shell command to manipulate the key database.
- You can use the syslog shell command to set the priority for the PPP syslog debug messages.

To run shell commands, you must access the command-interpreter mode. To access the command-interpreter mode from the kernel shell, enter the following command:

```
-> cmd
```

[vxWorks *} #

To return to the kernel shell, enter the following command:

```
# C
```

->

Next topic: About the pppconfig Shell Command on page 37

2. About the pppconfig Shell Command

The pppconfig shell command allows you to list and shutdown interfaces, display the Point-to-Point Protocol (PPP) configuration, and configure PPP parameters.

The multilink options appear only if the VxWorks Source Build (VSB) and VxWorks Image Project (VIP) have been built with PPP Multilink Protocol (MP).

To view the current configuration, use the show command. For example (default configuration):

```
# pppconfig show

PPP(oE) configuration:

[device]
ipppp.auth = noauth
ipppp.baudrate = 9600
ipppp.flags = proxyarp
```


WNDRVR

```
ipppp.ipcp.addr = 0.0.0.0
ipppp.ipcp.dstaddr = 10.1.4.1
ipppp.ipcp.pool = default
ipppp.ipcp.pool.default = 10.1.3.1-10.1.3.255
ipppp.lcp.echo_failure = 5
ipppp.lcp.echo_interval = 60
ipppp.lcp.mru = 1500
ipppp.lcp.mtu = 1500
ipppp.mp.enable = 0
ipppp.mp.frag_disable = 0
ipppp.mp.mrru = 1500
ipppp.mp.ssn = 0
ipppp.password = kallekula
ipppp.pppoe.ac_name = Windriver PPPoE Server
ipppp.pppoe.max_eth_sessions = 8
ipppp.pppoe.max_sessions = 16
ipppp.pppoe.mp_links = 1
ipppp.pppoe.secret_name = puttoanythinghere
ipppp.pppoe.server = 0
ipppp.pppoe.service_name = myservice
ipppp.runmode = start,passive,wincompat
ipppp.username = ppp

[interfaces]
ppp0.ipppp.ipcp.dstaddr = 10.1.4.1
```

To get help for the **pppconfig** command, use the following command:

```
# pppconfig
```

Using pppconfig to Show VJC Interface Information on page 38 When a link has successfully negotiated Van Jacobson compression (VJC), that information appears as a **use-vjc** flag in the output of the **pppconfig** command, in the driver information [drv] section of the output.
Using pppconfig to Manage PPP MP on page 39 Use the **pppconfig** command to manage Point-to-Point Protocol (PPP) Multilink Protocol (MP) at run-time.

Previous topic: About PPP Shell Commands on page 37
Next topic: Setting the PPP System Variables on page 41

2.1. Using pppconfig to Show VJC Interface Information

When a link has successfully negotiated Van Jacobson compression (VJC), that information appears as a **use-vjc** flag in the output of the **pppconfig** command, in the driver information [drv] section of the output.

About This Task

For example:

```
[drv]    baudrate=0  rcv-accm=0x0
 snd-accm=0x0  mru=1492  mtu=1492
            drv-flags=noflag noescape nofcs
noacfc use-vjc
```

Parent topic: About the pppconfig Shell Command on page 37

2.2. Using pppconfig to Manage PPP MP

Use the **pppconfig** command to manage Point-to-Point Protocol (PPP) Multilink Protocol (MP) at run-time.

About This Task

pppconfig mp-enable *<int>*

where *<int>* is 0 to disable or 1 to enable.

pppconfig mrru *<int>*

where <int> is the size of the Maximum Receive Reconstructed Unit.

pppconfig pppoe-links *<int>*

where *<int>* is the number of Point-to-Point Protocol over Ethernet (PPPoE) links to create when the interface is started.

Example PPP MP Runtime Configuration

The following command sequence creates a PPPoE bundle with two member links. In this example they are bound to the same Ethernet interface, **eth0**, but that is not required. Note that the command sequence assumes that a PPPoE server on a peer responds to both session requests.

Procedure

1. Set the PPP Multilink variables, if they are not already set:

```
# sysvar set -o ipppp.mp.enable 1
# sysvar set -o ipppp.pppoe.mp_links 2
```

2. Create the interfaces and bring them up:

```
# ifconfig pppoe0 create
# pppconfig pppoe0 pppoe-setif eth0
# ifconfig pppoe0 up
# ifconfig pppoe1 create
# pppconfig pppoe1 pppoe-setif eth0
# ifconfig pppoe1 up
```

Disabling PPP MP on page 39 Point-to-Point Protocol Multilink Protocol (PPP MP) can be disabled globally or on an individual interface.
Showing PPP MP Interface Information on page 40 Use the pppconfig command to show Point-to-Point Protocol Multilink Protocol (PPP MP) interface information.

Parent topic: About the pppconfig Shell Command on page 37

Disabling PPP MP

Point-to-Point Protocol Multilink Protocol (PPP MP) can be disabled globally or on an individual interface.

About This Task

To disable PPP Multilink globally:

```
# sysvar set -o ipppp.mp.enable 0
```

Note that the flag **-o** means to overwrite any existing setting.

To disable PPP Multilink on an interface use ppp0:

```
# sysvar set -c -o ipcom.if.ppp0.ipppp.mp.enable 0
```

Parent topic: Using pppconfig to Manage PPP MP on page 39

Showing PPP MP Interface Information

Use the pppconfig command to show Point-to-Point Protocol Multilink Protocol (PPP MP) interface information.

About This Task

In the output in the following example, the information that follows **[mp]** is specific to PPP Multilink and only appears if the PPP source code was built with IPPPP_USE_PPPMP.

For pppoe0, this example indicates that this is a bundle with two open links, a Maximum Receive Reconstructed Unit (MRRU) of 1500, and it is connected to the peer identified with Media Access Control (MAC) address 0:c0:a8:2:4:55.

For pppoe1, this example indicates that it is a member of a bundle with the same peer.

> **Note:** Member links do not have their own IP addresses; there is only one IP address for a bundle.

```
# pppconfig -a
pppoe0  PPPoE (eth0)
         inet 12.0.0.100  mask 255.255.255.255 peer 10.1.3.1
         inet6 unicast FE80::3C86:C8D3:8902:C573%pppoe0 peer
                                                FE80::4DF7:F67A:9C9:1AA3%pppoe0  prefixlen 128
         [netif]  mtu=1492 flags=UP RUNNING POINTOPOINT MULTICAST LINK1
         [ppp]    restart=3 max-failure=10 max-configure=10 max-terminate=2 max-authreq=3
                  flags=noaccomp noaccm
         [auth]   username=ppp password=kallekula  peer=<none>
         [lcp]    echo-interval=60  echo-failure=5
         [drv]    baudrate=0  rcv-accm=0x0 snd-accm=0x0  mru=1492  mtu=1492
                  drv-flags=noflag noescape nofcs noacfc
         [mp]     bundle, open-links=2
                  peer-epd-class=3 peer-epd-address=0:c0:a8:2:4:55 mrru=1500
```

```
 pppoe1  PPPoE (eth0)
         inet 0.0.0.0  mask 255.255.255.255 peer 0.0.0.0
         [netif]  mtu=1492 flags=RUNNING POINTOPOINT MULTICAST LINK1
         [ppp]    restart=3 max-failure=10 max-configure=10 max-terminate=2 max-authreq=3
                  flags=noaccomp noaccm <undef>
         [auth]   username=ppp password=kallekula  peer=<none>
         [lcp]    echo-interval=60  echo-failure=5
```


WNDRVR

```
        [drv]    baudrate=0  rcv-accm=0x0 snd-accm=0x0  mru=1492  mtu=1492
                 drv-flags=noflag noescape nofcs noacfc
        [mp]     member link
                 peer-epd-class=3 peer-epd-address=0:c0:a8:2:4:55 mrru=1500
```

Parent topic: Using pppconfig to Manage PPP MP on page 39

3. Setting the PPP System Variables

The sysvars have the property that if they are set in an unqualified way, they are global. However, if they are set as a user or interface specific variable, they are only used in that situation. This is true of all Point-to-Point Protocol (PPP) sysvars.

Global configuration parameters each correspond to a single sysvar. Per-interface and per-user configuration parameters correspond to lists of sysvars, one for each user or interface.

The configuration parameter for per-interface authentication modes is PPP_IF_AUTH_MODES_LIST. If per-interface authentication is set for the interfaces **ppp0** and **ppp1**, the corresponding sysvars would be ipcom.if.ppp0.ipppp.auth and ipcom.if.ppp1.ipppp.auth. The command formats are as follows:

- For per-interface sysvar, use ipcom.if.*ifname* **.***sysvarName*
- For per-user sysvar, use ipcom.users.*user* **.***sysvarName*

You can create user or interface specific values at run time using the **-c** flag with the sysvar command, for example:

```
# sysvar set -c ipcom.if.pppoe0.ipppp.mp.enable 1
```

This command will specifically enable MP on PPPoE interface **pppoe0**, even without **INCLUDE_IPPPP_INTERFACE_CONFIG**, and even if interface **pppoe0** has not been created yet.

The order of evaluation for sysvar is:

1. user specific
2. interface specific
3. global

For more information, see the ipppp_sysvar_get_conf( ) in **ipppp.c**.

The sysvars have precedence over variables established in the Workbench 4 kernel configuration editor.

For information on PPP sysvar, see the following sections:

- PPP Configuration Parameters on page 18
- PPP Interface Configuration Parameters on page 26
- PPP User Configuration Parameters on page 28
- PPPOE Configuration Parameters on page 29
- PPP Multilink Protocol Configuration Parameters on page 31
- PPP Van Jacobson Header Compression Configuration Parameters on page 34

For more information about sysvar, see the VxWorks 7 Network Stack Programmer's Guide.

WNDRVR

PPP Configuring an Additional Interface at Run Time on page 42 If you are not ready to configure the interface at build time, you can configure it at run time.
Configuring the IPCP Address Negotiation Cases on page 42 Internet Protocol Control Protocol (IPCP) allows two different methods of address negotiation. A peer can be configured to not solicit an address from another node, or a peer can be configured so that it uses a pool address to respond to a request from a peer for an address.
Configuring VJ Compression at Run-Time on page 43 You can configure Van Jacobson (VJ) compression at run-time.

Previous topic: About the pppconfig Shell Command on page 37
Next topic: Using the ifconfig Shell Command on page 43

3.1. PPP Configuring an Additional Interface at Run Time

If you are not ready to configure the interface at build time, you can configure it at run time.

About This Task

This procedure consists of two steps.

Procedure

1. Attaching a protocol.

   # **`ipAttach 1,"fei"`**

2. Configuring the address and subnet mask.

   # **`ifconfig "fei1 10.0.0.2 netmask 255.255.255.0 up"`**

   The parameters for the **ifconfig** command are specified in PPPConfiguring an Additional Interface at Build Time on page 35.

Parent topic: Setting the PPP System Variables on page 41

3.2. Configuring the IPCP Address Negotiation Cases

Internet Protocol Control Protocol (IPCP) allows two different methods of address negotiation. A peer can be configured to not solicit an address from another node, or a peer can be configured so that it uses a pool address to respond to a request from a peer for an address.

Configure a Peer to Not Solicit an Address

About This Task

Use the following commands and command formats to configure the peer so that it does not solicit an address from the other node:

- sysvar unset ipppp.ipcp.addr followed by sysvar set -c ipppp.ipcp.addr *ipv4-address* or
- sysvar set -o ipppp.ipcp.addr *ipv4-address*
- pppconfig addr *ipv4-address*

Use the following commands and command formats to configure the peer so that it uses the non-pool address to respond to a request from a peer for an address:

- sysvar unset ipppp.ipcp.dstaddr followed by sysvar set -c ipppp.ipcp.dstaddr *ipv4-address* or

- sysvar set -o ipppp.ipcp.dstaddr *ipv4-address*
- pppconfig dstaddr *ipv4-address*

Configure the Peer to Use a Pool Address

About This Task

Use the following commands and command formats to configure the peer so that it uses a pool address to respond to a request from a peer for an address:

- sysvar unset ipppp.ipcp.dstaddr
- sysvar unset ipppp.ipcp.pool.*pool-name* followed by sysvar set -c ipppp.ipcp.pool.*pool-name ipv4-address-range* or
- sysvar set -o ipppp.ipcp.*pool-name ipv4-address-range* where an example of an *ipv4-address-range* is *10.1.1.1-10.1.1.254*
- pppconfig setpool *pool-name start-ipv4-address end-ipv4-address*

Note: See the entry for PPP_IPCP_PEER_IPV4_ADDRESS_POOL in PPP Configuration Parameters on page 18for an explanation of ipppp.ipcp.pool.*pool-name.*

Parent topic: Setting the PPP System Variables on page 41

3.3. Configuring VJ Compression at Run-Time

You can configure Van Jacobson (VJ) compression at run-time.

About This Task

Example

To enable Van Jacobson (VJ) compression globally at run time use the following sysvar command:

```
# sysvar set -o ipppp.vjc.enable 1.
```

To set the VJC sysvar for interface **ppp0**, use the following sysvar command:

```
# sysvar set -o -c ipcom.if.ppp0.ipppp.vjc.enable 1
```

Parent topic: Setting the PPP System Variables on page 41

4. Using the ifconfig Shell Command

Many operations on Point-to-Point Protocol (PPP) interfaces are handled by the network command ifconfig.

A typical command sequence follows:

To create an interface:

```
# ifconfig   interface_name   create
```

To bring an existing interface up:

```
# ifconfig  interface_name   up
```

To shut an interface down:

```
# ifconfig  interface_name   down
```

To detach a pppoe interface

```
# ifconfig  interface_name   detach
```

To detach a ppp interface:

```
# ifconfig  interface_name   up down detach
```

Using ifconfig to Show PPP MP Interface Information

The ifconfig command also shows PPP Multilink information but only when the link is a bundle or a member of a bundle, as shown in the following example.

```
# ifconfig -a

 pppoe0  Link type:PPP over Ethernet  Queue:none
         PPP Multilink bundle
         inet 12.0.0.100  mask 255.255.255.255 peer 10.1.3.1
         inet6 unicast FE80::3C86:C8D3:8902:C573%pppoe0 peer
                                          FE80::4DF7:F67A:9C9:1AA3%pppoe0  prefixlen 128
         UP RUNNING POINTOPOINT MULTICAST LINK1
         MTU:1492  metric:2  VR:0  ifindex:4
         RX packets:0 mcast:0 errors:0 dropped:0
         TX packets:51 mcast:5 errors:0
         collisions:0 unsupported proto:0
         RX bytes:0  TX bytes:934

 pppoe1  Link type:PPP over Ethernet  Queue:none
         PPP Multilink member link
         inet 0.0.0.0  mask 255.255.255.255
 peer 0.0.0.0
         RUNNING POINTOPOINT MULTICAST LINK1
         MTU:1492  metric:2  VR:0  ifindex:5
         RX packets:0 mcast:0 errors:0 dropped:0
         TX packets:39 mcast:0 errors:0
         collisions:0 unsupported proto:0
         RX bytes:0  TX bytes:449
```

Viewing the State of the PPP Process on page 45 Use the ipd command to view the state of the PPP process (ipppp).

Previous topic: Setting the PPP System Variables on page 41
Next topic: Working with the Key Database on page 45

4.1. Viewing the State of the PPP Process

Use the ipd command to view the state of the PPP process (ipppp).

About This Task

For example:

```
# ipd list
Services:
...
ipppp     started
```

Parent topic: Using the ifconfig Shell Command on page 43

5. Working with the Key Database

Point-to-Point Protocol (PPP) supports for the secure key storage feature enabled through use of the Wind River Cryptography Libraries. The secure key database must be populated with the passwords before the PPP module can retrieve them.

This can be performed using either the secure key database APIs (see the **ipcom_key_db** library) or the secure key database shell command (keydb).

The IPCOM_KEY_DB_KEY_ENCRYPTING_KEY VIP configuration parameter value is used to encrypt keys in the key database. This parameter should be set to a random string of at least 40 characters, containing both lower and upper case, numbers and special characters. IPCOM_KEY_DB_KEY_ENCRYPTING_KEY has no default value.

Use the keydb import buf key_id *passwordOrPresharedKey* command to import a password or preshared key into the key database and assign it a **key_id** identifier. For more information on the keydb shell command, see VxWorks 7 Cryptography Libraries Programmer's Guide.

Previous topic: Using the ifconfig Shell Command on page 43
Next topic: Setting the PPP System Log Priority on page 45

6. Setting the PPP System Log Priority

Run the syslog list command to get the current priority level for all modules.

```
# syslog list
    syslog facility  priority
...
ipppp Error
```

To list the available syslog priority levels, use the following command:

```
# syslog priority *
syslog: missing <prio>
        priorities: Emerg Crit Error Warning Notice Info Debug Debug2
```


WNDRVR

Use the following command syntax to change the priority level for PPP:

```
# syslog priority ipppp priorityLevel
```

For more information on the syslog shell command, see the VxWorks 7 Network Stack Programmer's Guide.

Previous topic: Working with the Key Database on page 45
Next topic: Configuring a PPPoE Server at Run-time Over a VLAN Connection on page 46

7. Configuring a PPPoE Server at Run-time Over a VLAN Connection

You can configure Point-to-Point Protocol over Ethernet (PPPoE) at run-time rather than compile time over a VLAN connection.

About This Task

To configure the PPPoE server at run-time, do the following:

Procedure

1. Enable the node as a PPPoE server using the pppconfig pppoe-server 1 command or the sysvar set -o ipppp.pppoe.server 1 command.
2. Create the VLAN pseudo-interface using the ifconfig vlan *n* create command. A useful convention is to make the value of *n* the same as the value of the VLAN tag ID. You must create a VLAN pseudo-interface on the client that has the same VLAN tag ID.
   1. For the configuration shown in the figures titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN* and *Clients Determine Local and Peer PPP IP Addresses*, the command is ifconfig vlan21 create.
   2. For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the commands are the following:

      ifconfig vlan21 create

      ifconfig vlan22 create

3. Assign a VLAN tag ID to the VLAN pseudo-interface, associate the pseudo-interface with an Ethernet interface, assign a VLAN IP address to it and put it into service using the ifconfig vlan *n* vlan *m* vlanif *ethernet_interface vlan_ip_addr* up command.
   1. For the configuration shown in the figures titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN* and *Clients Determine Local and Peer PPP IP Addresses*, the command is ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.1/24 up.
   2. For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the commands are the following:

      ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.1/24 up

      ifconfig vlan22 vlan 22 vlanif eth0 10.10.22.1/24 up

4. Make any necessary modifications to the PPP IPCP parameters using the sysvar or pppconfig command.

   The configurations shown in the figures titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN* and *Clients Determine Local and Peer PPP IP Addresses* are an instance of *Case* 1 in the table titled *Configuring IPCP IP Address Negotiation Between Two Nodes* in PPP IPCP Configuration Parameters on page 23. See the discussion of this *Case* in PPP IPCP Configuration Parameters on page 23. The commands are the following:

   1. pppconfig addr 10.1.4.1 or sysvar set -o ipppp.ipcp.addr "10.1.4.1"

2. sysvar unset ipppp.ipcp.dstaddr
3. pppconfig setpool default 10.1.10.1 10.1.10.240 or sysvar set -o ipppp.ipcp.pool.default "10.1.10.1-10.1.10.240"

5. The configuration show in the figure titled *Clients Determine Local and Peer PPP IP Addresses* is an instance of *Case* 4 in the table titled *Configuring IPCP IP Address Negotiation Between Two Nodes* in PPP IPCP Configuration Parameters on page 23. See the discussion of this *Case* in PPP IPCP Configuration Parameters on page 23. No changes from the default are required to be made.

Results

Figure 4. Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN

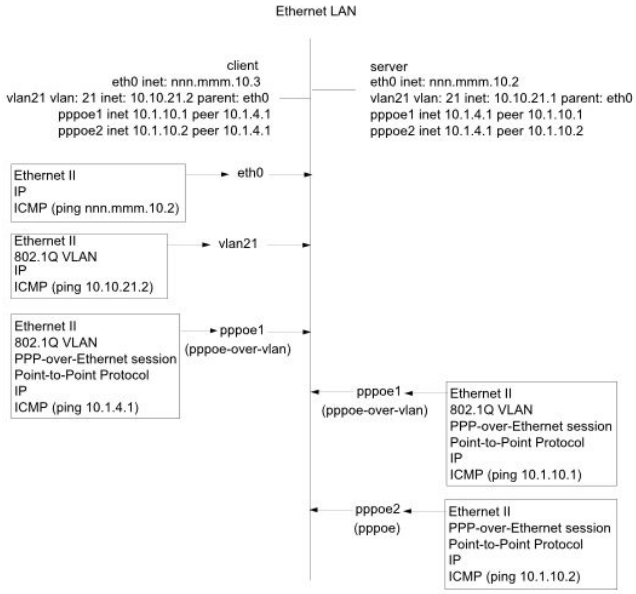


Figure 5. Clients Determine Local and Peer PPP IP Addresses

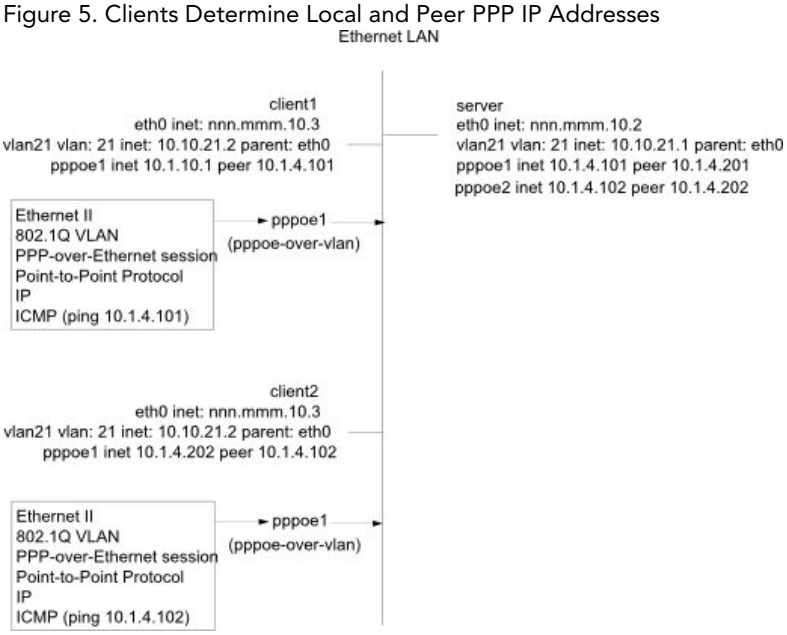
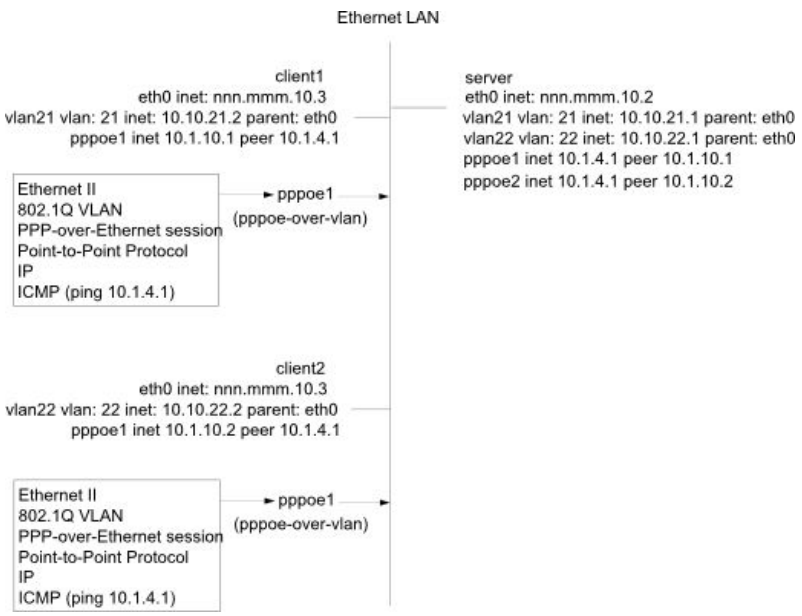


Figure 6. Multiple VLANs with Dynamic Address Assignment



Previous topic: Setting the PPP System Log Priority on page 45
Next topic: Configuring a PPOE Client at Run-time over a VLAN Connection on page 48

8. Configuring a PPOE Client at Run-time over a VLAN Connection

You can configure Point-to-Point Protocol over Ethernet (PPPoE) client at run-time rather than compile time over a VLAN connection.

About This Task

To configure the client at run-time, do the following:

Procedure

1. Create the VLAN pseudo-interface using the ifconfig vlan *n* create command. A useful convention is to make the value of *n* the same as the value of the VLAN tag ID.
   1. For the configuration shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN*, the command is ifconfig vlan21 create.
   2. For the configuration shown in the figure titled *Clients Determine Local and Peer PPP IP Addresses* and the figure titled *Multiple VLANs with Dynamic Address Assignment*, the command is ifconfig vlan21 create on both clients.
2. Assign a VLAN tag ID to the VLAN pseudo-interface, associate the pseudo-interface with an Ethernet interface, assign a VLAN IP address to it and put it into service using the ifconfig vlan *n* vlan *m* vlanif *ethernet_interface vlan_ip_addr* up command. The value of *m*, the VLAN tag ID, must be the same as the one on the server, and the *vlan_ip_addr* must be on the same subnet as the *vlan_ip_addr* assigned to the corresponding VLAN pseudo-interface on the server.

   For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the commands are ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.2/24 up on client1 and ifconfig vlan22 vlan 22 vlanif eth0 10.10.22.2/24 up on client2

   1. For the configuration shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN*, the command is ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.2/24 up..

2. For the configuration shown the figure titled *Clients Determine Local and Peer PPP IP Addresses*, the commands are ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.2/24 up on client1 and ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.3/24 up on client2.
3. For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the commands are ifconfig vlan21 vlan 21 vlanif eth0 10.10.21.2/24 up on client1 and ifconfig vlan22 vlan 22 vlanif eth0 10.10.22.2/24 up on client2.

3. Make any necessary modifications to the PPP IPCP parameters using the sysvar or pppconfig command.
   1. The configurations shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN* and the figure titled *Multiple VLANs with Dynamic Address Assignment* are an instance of Case 1 in the table titled *Configuring IPCP IP Address Negotiation Between Two Nodes*. See the discussion of this case in PPP IPCP Configuration Parameters on page 23. No change in the client's configuration is required.
   1. The configuration shown in the figure titled *Clients Determine Local and Peer PPP IP Addresses* is an instance of Case 4 in the table titled *Configuring IPCP IP Address Negotiation Between Two Nodes*. See the discussion of this case in PPP IPCP Configuration Parameters on page 23. The commands are the following:

   pppconfig addr 10.1.4.201 . command or a sysvar set -o ipppp.ipcp.addr "10.1.4.201" command to make the local address static.

   pppconfig dstaddr 10.1.4.101 command or a sysvar set -o ipppp.ipcp.dstaddr "10.1.4.101" command to provide an address that is given to the peer when it is requested during the IPCP phase of the PPP call setup.

4. Create the PPPoE interface using the ifconfig pppoe *n* create command.
   1. For the configuration shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN*, the commands are ifconfig pppoe1 create for the PPPoE-over-VLAN interface and ifconfig pppoe2 create for the PPPoE interface.
   2. For the configurations shown in the figure titled *Clients Determine Local and Peer PPP IP Addresses*, the command is ifconfig pppoe1 create on each client.
5. Set the interface used by the pppoe interface to be the VLAN pseudo-interface. Use the command pppconfig pppoe *n* pppoe-setif *interface-name* .

   For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the command is pppconfig pppoe1 pppoesetif vlan21 on client1 and pppconfig pppoe1 pppoesetif vlan21 on client2.

   1. For the configuration shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN*, the command is pppconfig pppoe1 pppoe-setif vlan21 for the PPPoE-over-VLAN interface and pppconfig pppoe2 pppoe-setif eth0 for the PPPoE interface.
   2. For the configuration shown in the figure titled *Clients Determine Local and Peer PPP IP Addresses*, the command is pppconfig pppoe1 pppoesetif vlan21 on each client.
   3. For the configuration shown in the figure titled *Multiple VLANs with Dynamic Address Assignment*, the command is pppconfig pppoe1 pppoesetif vlan21 on client1 and pppconfig pppoe1 pppoesetif vlan21 on client2.
6. Put the PPPoE interface into service using the ifconfig pppoe *n* up command.
   1. For the configuration shown in the figure titled *Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN*, the commands are ifconfig pppoe1 up and ifconfig pppoe2 up.
   2. For the configuration shown in the figure titled *Clients Determine Local and Peer PPP IP Addresses* and the figure titled *Multiple VLANs with Dynamic Address Assignment*, the command is ifconfig pppoe1 up on each client.

Results

Figure 7. Client and Server Connected by Ethernet, VLAN and PPPoE over VLAN

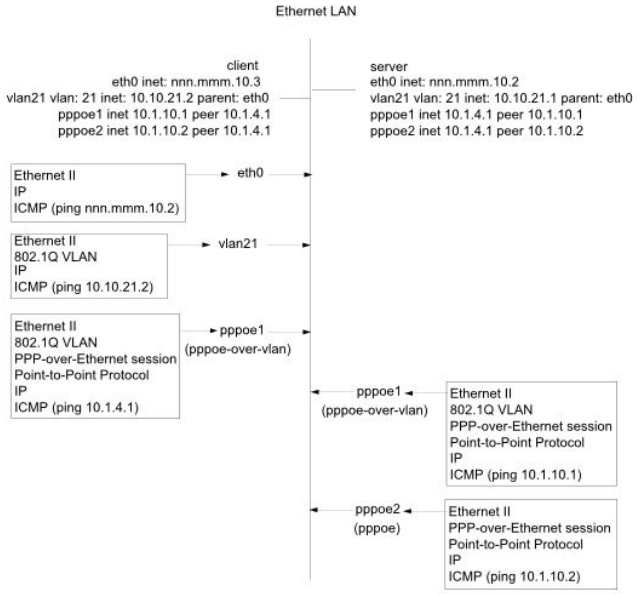


Figure 8. Clients Determine Local and Peer PPP IP Addresses

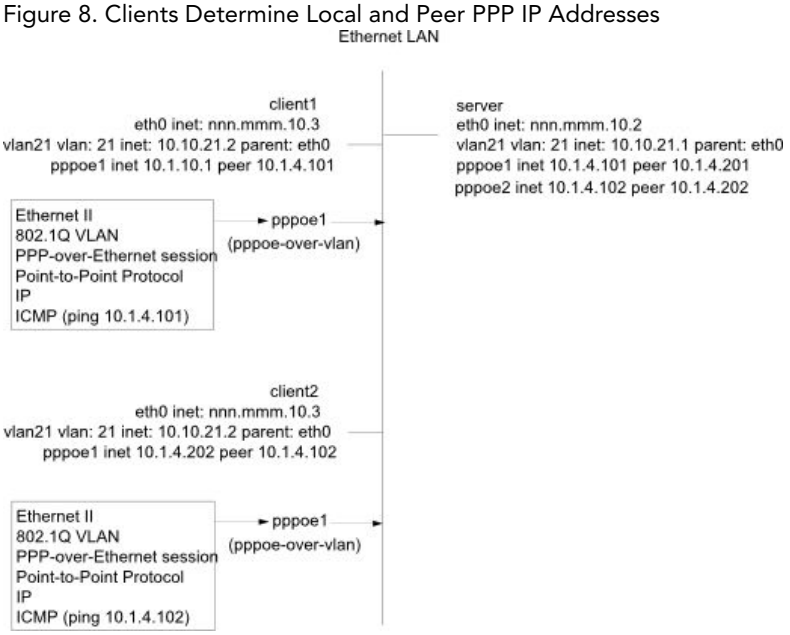
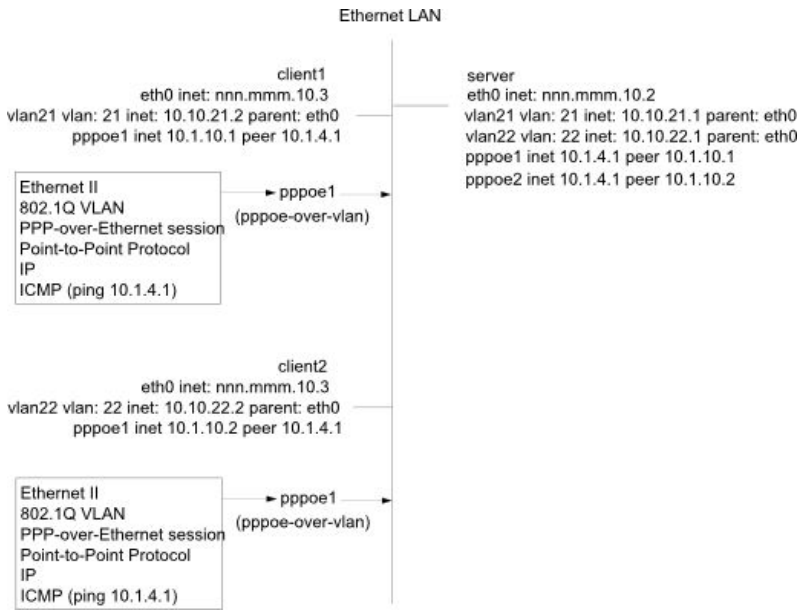


Figure 9. Multiple VLANs with Dynamic Address Assignment



Previous topic: Configuring a PPPoE Server at Run-time Over a VLAN Connection on page 46

5. API FUNCTIONS

1. Callback Function

The **ipppp** library contains a single function that handles Point-to-Point Protocol (PPP) actions (ipppp_example_action_cb( )).

PPP requires a callback function to handle all of the following actions:

IPPPP_ACTION_INIT

Called when a new interface is attached. Use this callback to set configuration parameters for the interface, like the restart timeout or max-failure settings in the Ipcom_ppp_conf struct.

IPPPP_ACTION_LCP_STARTING

Called when LCP starts.

IPPPP_ACTION_IPCP_STARTING

Called when IPCP starts.

IPPPP_ACTION_IPV6CP_STARTING

Called when IPV6CP starts.

IPPPP_ACTION_IP_UP

Called when IPV4 is up and ready for traffic.

IPPPP_ACTION_IP_DOWN

Called when IPv4 goes down.

IPPPP_ACTION_IPV6_UP

Called when IPv6 is up and ready for traffic.

IPPPP_ACTION_IPV6_DOWN

Called when IPv6 goes down.

IPPPP_ACTION_PEER_IDENTIFIER

Called when a peer sends an IPv6 Identifier-Configure request. Return 0 to accept the request, -1 to deny the request.

IPPPP_ACTION_LOGIN

Called when authentication (username and password) is required.

To define a callback function and register it with Wind River PPP, set the parameters PPP_INSTALL_CALLBACK_HOOK and PPP_ACTION_CALLBACK_HOOK. When PPP_INSTALL_CALLBACK_HOOK is set to TRUE, the user-defined callback function identified by PPP_ACTION_CALLBACK_HOOK is registered. If a user-defined callback function is not added, the default callback function in **ipppp_example.c** is used.

For more information, see **ipppp_example.c** and the reference entry for ipppp_example_action_cb( ).

6. IOCTLS

1. ioctl( ) Requests

Use these ioctl( ) requests to get and set driver and link layer data.

Next topic: ioctl( ) Data Structures on page 53

2. ioctl( ) Data Structures

Important data types for Point-to-Point Protocol (PPP) ioctl( ) requests are defined in **ipppp.h**.

All ioctl( ) requests use the **Ip_pppreq** struct for passing data. The **pppr_name** member must be set to the interface name. Different requests use different members of the **pppru** union. See Driver ioctl( ) Requests on page 58 and Link Layer ioctl( ) Requests on page 58 for details.

Ip_pppreq

```
struct Ip_pppreq
{
      char pppr_name[IP_IFNAMSIZ]; /* if name, e.g. "ppp0" */
      int link_index; /* link_index, currently unused (for MPPP) */

      union
      {
          Ip_u32 ppp_flags; /* Get/Set PPP flags. */
          Ipcom_ppp_conf pppconf; /* Get/Set PPP protocol configuration. */

          char username[IPCOM_AUTH_USERNAME_MAX];
          char password[IPCOM_AUTH_PASSWORD_MAX];

          char if_name[IP_IFNAMSIZ];

          Ipcom_ppp_drvconf drvconf; /* Get PPP driver configuration. */
          Ip_u32 drv_baudrate; /* Set PPP driver baudrate. */
          int drv_wincompat; /* Windows compat mode */
          Ip_u32 drv_rcv_accm; /* Set PPP driver Recv ACCM. */

          Ipcom_ppp_debuginfo debuginfo; /* Get PPP debug info. */
          Ipcom_ppp_login login; /* Set PPP login info. */
          Ipcom_ppp_auth auth; /* Set PPP authenticate info. */
  #ifdef IPPPP_USE_PPPMP
          Ip_u32 pppflags; /* PPP flags for the I/F (ppp_flags are the link) */
          int open_links; /* Number of links in use by ppp multilink */
  #endif

  #ifdef IPPPP_USE_PPPL2TP
          char *pppl2tp_paddr_str; /* IP address of LNS to request tunnel with */
          Ipppp_pppl2tp_drv_link pppl2tp_drv_link;
  #endif
```

```
      }
      pppru;
  };

  typedef struct Ipcom_ppp_drvconf_struct
  {
      Ip_u32 drv_flags; /* PPP driver flags. */
      Ip_u32 baudrate; /* Baudrate. */

      Ip_u32 rcv_accm; /* Recv asyncmap. Default 0xffffffff */
      Ip_u32 req_rcv_accm; /* Recv asyncmap request, to negotiate if possible */
      Ip_u32 snd_accm; /* Send asyncmap. Default 0xffffffff */
      Ip_u16 mru; /* Maximum Receive Unit. Default 1500. */
      Ip_u16 mtu; /* Maximum Transmit Unit. Default 1500. */

  #ifdef IPPPP_USE_PPPMP
      Ip_u16 local_mrru;
      Ip_u16 peer_mrru;
      Ip_u8 peer_epd_class; /* Endpoint discriminator class */
      Ip_u8 peer_epd_data[IPPPPMP_EPD_DATA_MAX]; /* Endpoint discriminator data */
      Ip_u8 peer_epd_datalen;      Ip_u8 ssn; /*Short Sequence Number usage allowed (1) or not (0
)*/
  #endif /* IPPPP_USE_PPPMP */
  }
  Ipcom_ppp_drvconf;
```

Ipcom_ppp_conf

The **Ipcom_ppp_conf** struct is used for link layer configuration data. These values are set to default when the interface is first initialized. To change any of the default values use an **IP_SIOCXPPPSCONF** ioctl( ) request in the action callback function. See Callback Function on page 52 for details.

```
typedef struct Ipcom_ppp_conf_struct
{
     Ip_u16 restart;
     Ip_u8  max_failure;
     Ip_u8  max_configure;
     Ip_u8  max_terminate;
     Ip_u8  max_authreq;
     Ip_u8  pad[2];
}
Ipcom_ppp_conf;
```

The struct members are as follows:

restart

Sets the PPP Configuration/Termination Request retransmission timeout in seconds. Default 3.

max_failure

Sets the maximum number of PPP configuration negative-acknowledges (NAKs) returned before sending Configuration Reject. Default 10.

max_configure

Sets the maximum number of PPP configuration request retransmissions. Default 10.

WNDRVR

max_terminate

Sets the maximum number of PPP termination request retransmissions. Default 2.

max_authreq

Specifies the maximum number of PPP authentication retransmissions to be made before the link terminates. Default 3.

Ipcom_ppp_drvconf

The **Ipcom_ppp_drvconf** struct is used for driver configuration data.

```
typedef struct Ipcom_ppp_drvconf_struct
{
    Ip_u32     drv_flags;     /* PPP driverflags. */
    Ip_u32     baudrate;      /* Baudrate.*/
    Ip_u32     rcv_accm;      /* Recv asyncmap. Default 0xffffffff */
    Ip_u32     snd_accm;      /* Send asyncmap. Default 0xffffffff */
    Ip_u16     mru;           /* Maximum Receive Unit. Default 1500. */
    Ip_u16     mtu;           /* Maximum Transmit Unit. Default 1500. */
    Ip_u16     local_mrru;    /*local Maximum Received Reconstructed Unit */
    Ip_u16     peer_mrru;     /*peer Maximum Received Reconstructed Unit */
    Ip_u8      peer_epd_class; /*peer endpoint class */
    Ip_u8      peer_epd_data[IPPPPMP_EPD_DATA_MAX];  /* peer endpoint data */
    Ip_u8      peer_epd_datalen; /* peer
endpoint data length */
}
Ipcom_ppp_drvconf;
```

The struct members are as follows:

drv_flags

The PPP driver flags. These flags cannot be modified, since they define the nature of the PPP driver.

The available flags are:

IP_DRVPPP_FLAG_NOFLAG

No initial/trailing PPP flag (0x7e).

IP_DRVPPP_FLAG_NOESCAPE

Do not byte-escape output/input packets.

IP_DRVPPP_FLAG_NOFCS

Do not input-check or add output FCS.

IP_DRVPPP_FLAG_NOACFC

Do not send or receive addr(0xff) ctrl(0x03).

baudrate

The current PPP RS232 serial driver baudrate.

rcv_accm

The current PPP receive asynchronous control character map (ACCM) bit-field.

WNDRVR

snd_accm

The current PPP send asynchronous control character map (ACCM) bit-field.

mru

The current driver maximum receive unit (MRU).

mtu

The current driver maximum transmit unit (MTU).

local_mrru

The local Maximum Receive Reconstructed Unit (MRRU).

peer_mrru

The peer's Maximum Receive Reconstructed Unit (MRRU).

peer_epd_class

The peer's endpoint discriminator (EPD) class.

peer_epd_data[IPPPPMP_EPD_DATA_MAX]

The peer's endpoint discriminator data size.

peer_epd_datalen

The peer's endpoint discriminator data length.

Ipcom_ppp_debuginfo

The **Ipcom_ppp_debuginfo** struct contains debugging data.

```
typedef struct Ipcom_ppp_debuginfo_struct
{
    Ip_u32  link_flags;  /* See ipnet_netif.h */
    Ip_u32  pppoe_eth_ifindex;
    Ip_u8   link_state;
    Ip_u8   lcp_state;
    Ip_u8   auth_state;
    Ip_u8   ipcp_state;
    Ip_u8   ipv6cp_state;
}
Ipcom_ppp_debuginfo;
```

Ipcom_ppp_login

This is the ioctl structure for **IP_SIOCXPPPLOGIN**. It is used to login a user during PPP PAP/CHAP negotiation.

```
typedef struct Ipcom_ppp_login_struct
{
    int   ifindex;
    int   link_index;
    void *login_cookie;
    char  username[IPCOM_AUTH_USERNAME_MAX];
    char  password[IPCOM_AUTH_PASSWORD_MAX];
```

```
}
Ipcom_ppp_login;
```

The struct members are as follows;

ifindex

PPP interface index

link_index

PPP link index (currently always 0)

login_cookie

IPPPP specific, do not read/write. Pass the same cookie from the action callback to the login function.

username

PPP username, for example, "bill"

password

The cleartext password for the user named in **username**.

Ipcom_ppp_auth

This is the ioctl structure for IP_SIOCXPPPAUTH. It is used to authenticate a user during PPP PAP/CHAP negotiation.

```
typedef struct Ipcom_ppp_auth_struct
{
    int  ifindex;
    int  link_index;
    Ip_bool success;
    Ipcom_auth_userinfo  userinfo;
}
Ipcom_ppp_auth;
```

The struct members are as follows:

ifindex

PPP interface index

link_index

PPP link index (currently always 0)

login_cookie

IPPPP specific, do not read/write. Pass the same cookie from the action callback to the login function.

user_info

This is user information filled in by the login functions.

Previous topic: ioctl( ) Requests on page 53
Next topic: Driver ioctl( ) Requests on page 58

3. Driver ioctl( ) Requests

Use driver ioctl( ) requests to get and set driver data.

ioctl( )	Ip_pppreq.pppru member	Description
IP_SIOCXPPPGDRVCONF	drvconf	Get current driver configuration.
IP_SIOCXPPPGDRVINFO	drvconf	Get current driver configuration.
IP_SIOCXPPPSDRVBAUDRATE	drv_baudrate	Set the driver baud rate.
IP_SIOCXPPPSDRVUP		Bring the driver up. For testing purposes.
IP_SIOCXPPPSDRVDOWN		Shut the driver down. For testing purposes.
IP_SIOCXPPPSDRVWINCOMPAT	drv_wincompat	Set the driver to Windows compatibility mode. See the **PPP_RUNMODE** parameter in PPP Configuration Parameters on page 18 for details.

Previous topic: ioctl( ) Data Structures on page 53
Next topic: Link Layer ioctl( ) Requests on page 58

4. Link Layer ioctl( ) Requests

Use link layer ioctl( ) requests to get and set link data.

ioctl( )	Ip_pppreq.pppru member	Description
IP_SIOCXPPPGFLAGS	**ppp_flags**	Bit-mask of the following flags: **IP_PPP_FLAG_NOACCOMP** Disable Address/Control compression in both directions (snd/rcv).
		IP_PPP_FLAG_NOACCM Do not send or ack ACCM confreq. **IP_PPP_FLAG_AUTH** Require the peer to authenticate itself before allowing snd/rcv NCP.

WNDRVR

ioctl( )	Ip_pppreq.pppru member	Description
		IP_PPP_FLAG_NOAUTH Do not require the peer to authenticate itself. That is, do not ask for authentication. **IP_PPP_FLAG_REFUSE_PAP** Do not agree to authenticate using PAP. **IP_PPP_FLAG_REFUSE_CHAP** Do not agree to authenticate using MD5 CHAP.
IP_SIOCXPPPSFLAGS	**ppp_flags**	See **IP_SIOCXPPPGFLAGS**.
IP_SIOCXPPPGCONF	**pppconf**	Get the PPP main configuration.
IP_SIOCXPPPSCONF	**pppconf**	Set the PPP main configuration.
IP_SIOCXPPPGUSER	**username**	Get the local user.
IP_SIOCXPPPSUSER	**username**	Set the local user.
IP_SIOCXPPPGPASSWD	**password**	Get the local password.
IP_SIOCXPPPSPASSWD	**password**	Set the local password.
IP_SIOCXPPPGPEERUSER	**username**	Get the peer user.
IP_SIOCXPPPSETIF	**if_name**	Set the PPPoE Ethernet interface.
IP_SIOCXPPPGDINFO	**debuginfo**	Get the PPP debug info.
IP_SIOCGPPPIFFLAGS	**pppflags**	Get the interface flags, which contain flags for multilink bundle membership.
IP_SIOCXPPPLOGIN	**login**	Set PPP login info.
IP_SIOCXPPPAUTH	**auth**	Set PPP authenticate info.
IP_SIOCGPPPIFLINKS	**open_links**	Get the number of open links currently in a bundle.

Previous topic: Driver ioctl( ) Requests on page 58

7. PPP APPLICATIONS

1. About PPP Applications

Applications that use Point-to-Point Protocol (PPP) must manage the PPP interfaces and PPP events.

Next topic: PPP and Serial Interfaces on page 60

2. PPP and Serial Interfaces

Point-to-Point Protocol (PPP) integrates directly with the Wind River Network Stack. If PPP is enabled, the PPP protocol is automatically attached to serial interfaces when they are brought up.

This creates a PPP interface named **ppp***n*, where *n* is an assigned interface number starting from 0. For example, the first PPP interface is named **ppp0**.

In addition, if the run mode for the interface is set to "start", PPP starts automatically on the attached interface. To start PPP on an interface that is not set to start automatically, issue an **IP_SIOCSIFFLAGS** ioctl( ) request with the **IP_IFF_UP** bit set. See **ipppp_cmd_pppconfig.c** for examples of getting and setting interface flags using **IP_SIOCGIFFLAGS** and **IP_SIOCSIFFLAGS**.

PPP on a Target with a Single Serial Device

The default behavior of a VxWorks image is based on the assumption that the target has 2 serial devices. The console is assigned to the first serial device whose name is **/ttyS0**, and the **ppp0** interface is assigned to the second serial device whose name is **/ttyS1**. PPP requires at least one serial device, typically available through a 9-pin DE-9 or similar connector. Wind River PPP does not run on USB. For more information on serial device support, consult the VxWorks BSP References for the target hardware.

The console and the PPP protocol cannot run over the same link. If you want to use PPP on a target with a single serial device, you must redirect the console and reassign PPP to the single serial device. The redirection of the console and the reassignment of the **ppp0** interface to **/ttyS0** is done at build time. At run time you can telnet to the console-less node to run commands.

Debugging PPP on page 60
PPP LCP Echo Requests Lead to Connection Termination on page 62

Previous topic: About PPP Applications on page 60
Next topic: Guidelines for Using PPP on Custom Serial Devices on page 63

2.1. Debugging PPP

VxWorks Source Build (VSB)

The default VxWorks source build (VSB) can be used for debuging because **COMPONENT_IPPPP** is enabled by default. If you wish to enable debugging, modify the following in the VSB:

- **Set the IPNET_BUILD_MODE_CHOICE** to **debug mode**
- **IPCOM_DEFAULT_SYSLOG_PRIORITY_DEBUG** to **y**

VxWorks Image Project (VIP)

Redirecting the console is done by changing the value of the CONSOLE_NAME parameter from **ttyS0** to **ttyS2**. This parameter belongs to the INCLUDE_SIO component. Redirection means that console traffic is not occurring on the serial device that is now going to be used for the **ppp0** interface.

Reassigning the ppp0 interface to /ttyS0 is done by modifying a line of code in **ipppp_config.c** in the **ipppp_conf_sysvar** array so that the **ipcom.if.ppp0.devname** sysvar is mapped to **/ttyS0** instead of **/ttyS1**. The unmodified line of code in question is the following:

```
{ "ipcom.if.ppp0.devname", "/ttyS1" }
```

Add INCLUDE_IPPPP and INCLUDE_IPPPP_CMD in the VxWorks image project (VIP).

If you configure syslog to output network stack actions at the debug level to a file in /ram, you can browse the file using the more command to verify that the PPP input daemon was started on **/ttyS0**.

To enable the debugging of this scenario modify the following in the VIP:

- Add INCLUDE_IPCOM_SYSLOGD_CMD and set the IPCOM_SYSLOGD_DEFAULT_PRIORITY parameter to **IPCOM_LOG_DEBUG**
- Add **INCLUDE_IPCOM_SYSLOGD_USE_LOG_FILE**. Do not change the IPCOM_SYSLOGD_LOG_FILE parameter. Its default value will cause a file called syslog to be written to the **IPCOM_FILE_ROOT** directory on the target (typically **/ram**).

To see the syslog file in the file system and browse it, add **INCLUDE_DISK_UTIL** and .

To support telnet access add **INCLUDE_IPTELNETS**.

Add **INCLUDE_IPIFCONFIG_CMD** to verify the **ppp0** interface is present, **INCLUDE_IPPING_CMD** to verify connectivity between peer nodes and **INCLUDE_IPCOM_SYSVAR_CMD** to see the value of the **ipcom.if.ppp0.devname, ipcom.if.ppp0.ipppp.ipcp.addr**, and **ipcom.if.ppp0.ipppp.ipcp.dstaddr** system variables.

Run Time

Once the telnet session to the target has been established, do the following:

```
    ->
cmd
    # ls /ram
    # more /ram/syslog
    # ifconfig -a
```

The **syslog** file will contain lines like the following:

```
    Debug: ipcom_drv_ppp0_input :: starting PPP input daemon on /ttyS0
    Info: [ppp0 sec=0 lcp=9 auth=9 ipcp=9] IPCP IPv4 UP - local 1.1.1.1, peer 2.2.2.2
```

The **ifconfig** output will show lines like the following:

```
    ppp0    Link type:Point to point Queue:none
            inet 1.1.1.1  mask 255.255.255.255 peer 2.2.2.2
```


WNDRVR

Example Verification Setup

This example verification setup uses a host and 2 targets. Each target has 1 DE-9 connector. The test requires 2 images. Both images are based on the same VSB. The two VIPS are configured with same components and parameter values. The only difference between the two is in the value of two system variables set in the **ipppp_config.c** file prior to building the VIP. The setting of these system variables satisfies the requirement that some way of assigning IP addresses to the two peers be used during the IPCP phase of the connection setup. For more information, see PPP IPCP Configuration Parameters on page 23.

Before building the first VIP set the variables in the **ipppp_config.c** file in the **ipppp_conf_sysvar** array as follow:

```
{ "ipcom.if.ppp0.ipppp.ipcp.addr", "1.1.1.1" },
{ "ipcom.if.ppp0.ipppp.ipcp.dstaddr", "2.2.2.2" }
```

Before building the second VIP set the variables in the **ipppp_config.c** file in the **ipppp_conf_sysvar** array as follow:

```
{ "ipcom.if.ppp0.ipppp.ipcp.addr","2.2.2.2" },
{ "ipcom.if.ppp0.ipppp.ipcp.dstaddr", "1.1.1.1" }
```

Actually, for this example, setting **ipcom.if.ppp0.ipppp.ipcp.addr** alone to a different IP address for each VIP build would be sufficient.

The host and the 2 targets are connected to the internet. The 2 targets have static IP addresses. The host contains the 2 VIP images and is running an FTP server. The host is connected to each target's serial port using PuTTY. The boot parameters for each target are set. The boot parameters record the host's IP address, the path to the image on the host, the FTP account username and password, and the target's IP address among others. The PuTTY sessions are terminated, and the host disconnected from the targets. The serial port of one target is connected to the serial port of the other target. The targets are powered down and then powered up. After allowing some time for the targets to initialize, two telnet sessions are started on the host, one to each target.

Parent topic: PPP and Serial Interfaces on page 60

2.2. PPP LCP Echo Requests Lead to Connection Termination

Trigger Event

A Point-to-Point Protocol (PPP) client is attempting to establish a PPP connection to a PPP server.

Behavior

You see the following behavior:

- PPP Link Control Protocol (LCP) configuration has completed.
- PPP IPCP configuration is in progress.
- A number of PPP LCP Echo Request packets interleave the PPP IPCP packets.
- The client sends an LCP termination request packet to the server before the connection has been fully established and any data has been sent.

Possible Cause

The PPP LCP Echo Request packets perform an "Are you there?" function. The PPP connection has not yet been established. The server does not respond to the Echo Request packets and the client requests the termination of the connection.

WNDRVR

Investigation

In the packet sniffer determine the interval in seconds between PPP LCP Echo Request packets and the number of PPP LCP Echo Request packets that occur prior to the sending of the termination request packet.

The interval value is configured at VIP build time using the PPP_LCP_ECHO_REQ_INTERVAL parameter. The default is 60 seconds, which is usually long enough to prevent this behavior. Verify the run-time value by executing sysvar list ipppp in the console and look for the value of **ipppp.lcp.echo_interval** and increase it using the following command format:

```
# sysvar set -o ipppp.lcp.echo_interval num
```

The number of Echo Request packets sent before a disconnect is requested is configured at VIP build time using the PPP_LCP_ECHO_REQ_FAILURE parameter. The default is 5. Verify the run-time value by looking for the value of **ipppp.lcp.echo_failure** in the **sysvar list ipppp** command output. Modify it using the command sysvar set -o ipppp.lcp.echo_failure *num* .

Resolution

To alter the pertinent values at runtime, you can use either the sysvar command or the pppconfig command.

The appropriate sysvar command formats are as follows:

```
# sysvar set -o ipppp.lcp.echo_interval num
# sysvar set -o ipppp.lcp.echo_failure num
```

The appropriate pppconfig command formats are as follows:

```
# pppconfig echo-interval  num
# pppconfig echo-failure  num
```

Verify the effect on the PPP connection. Once you are getting the behavior that you desire, you can set the build time parameters and rebuild the image.

Parent topic: PPP and Serial Interfaces on page 60

3. Guidelines for Using PPP on Custom Serial Devices

Point-to-Point Protocol (PPP) includes support for running the point-to-point protocol as described in RFC 1661 on VxWorks serial devices.

A custom serial device is an input/output device other than those that VxWorks already supports. If VxWorks already supports the device, no code changes should be required. The device can use any physical transmission method to communicate with a peer, but it must be serial in nature.

The custom serial device driver must meet the following requirements:

- The custom serial device must provide input and output operations for receiving and sending data. The PPP driver code extracts data for transmission from the packet structures used by the network stack, and formats received data into packets for passing to the network stack. The custom serial device must have input and output operations must be capable of handling this data.

- If the input operation of the custom serial device is blocking (as is, for example, a read operation on a serial device), then a means of interrupting the input operation must be provided so that when the system requests that the connection be dropped, the device can closed properly.
- Output operations should be non-blocking, so that the calling function returns immediately, without waiting for the actual output to take place, though if output operations happen quickly, this might not be required.
- For most devices, open and close operations are also required, where an open function prepares the device for use, and the close function releases resources. If the custom serial device has these requirements, then appropriate functions must be supplied.

Generally, a device driver for the custom serial device would provide these capabilities. Developing device drivers is beyond the scope of this chapter.

Overview

The VxWorks implementation of PPP driver for serial devices is recommended as a starting point for developing custom PPP drivers. This code, which can be found in **/net/ipnet/coreip-***x* **.***x* **.***x* **.***x* **/src/ipcom/src/ipcom_drv_ppp.c**, supplies support for VxWorks serial devices. This section discusses the structure of this file and how it can be extended to other serial devices.

The VxWorks PPP driver includes functions that network stack uses to control driver operation and to send packets. The code is designed to make use of the data structures found in the Wind River network stack, such as the network interface data structure (**Ipcom_netif**) and packet data structure (**Ipcom_pkt**). It also includes tasks and data structures specific to link-level operation.

The developer wishing to extend this code to custom serial devices should choose either to add to existing support, or replace it. To add a new device type, code relevant to this device must be added where the serial device is referred to, and code must be added to detect which type of device should be used. This could be done, for example, based on a device name; the **devname** member of the **Ipcom_drv_ppp_link** structure stores the device name. Alternatively, the code for handling a custom serial device could replace the existing serial device. This might be a simpler task, but if this path is chosen, PPP support for VxWorks serial devices will be lost. This discussion assumes that adding (the first option) is followed; to follow the second option, instead of adding, just replace the existing device handling code.

An additional implementation option would be to add a new interface type for the custom serial device. This would allow customization of more aspects of the network interface, including, for example, defining a new interface name other than the **ppp** used for PPP interfaces. However, adding a new interface type is a complex task and is beyond the scope of this document.

Interface Creation

A device name is associated with an interface at the time that the interface is created. For an interface using a custom serial device the device name should indicate the custom device. It should not be a VxWorks serial device name (**/ttyS0**), otherwise a VxWorks serial device would be associated with the interface.

The interface can be created either programmatically, or from the command shell. Device names are associated with interface names using system variables (sysvars).

For example, to associate custom device **custom0** with a yet-to-be created interface named **ppp5**, enter the following command in the ipcom shell to create the appropriate sysvar:

```
# sysvar set -co ipcom.if.ppp5.devname custom0
```

To create an interface from the shell prompt use the following command:

```
# ifconfig create ppp5
```

This command will result in the **ipcom_drv_if_init** function being called with an interface name of **ppp5** and a device name of **custom0**. To create an interface programmatically, use the **IP_SIOCIFCREATE** ioctl, which will also result in **ipcom_drv_if_init** being called. The following code, for example, could be used when a new interface is needed:

WNDRVR

```
Ip_fd fd;
struct Ip_ifreq ifreq;

fd = ipcom_socket(IP_AF_INET, IP_SOCK_DGRAM,0);
ipcom_strncpy(ifreq.ifr_name, "ppp5", IP_IFNAMSIZ);
ipcom_socketioctl(fd, IP_SIOCIFCREATE, &ifreq);
```

This will have the same result as the **ifconfig create** command. In both cases the device name will be stored in the **devname** member of the **Ipcom_drv_ppp_link** structure, where it will be accessible from the network interface structure.

Device Open

The function **ipcom_drv_ppp_open** is called when a serial device needs to be opened. This function is called when the ifconfig up command is entered, or the **IP_SIOCXOPEN** ioctl is called for a PPP interface. To add support for a custom serial device, this function should compare the name of the device to the custom device name, and, if matching, using the open procedure for the custom serial device. For example, if the custom serial device names start with the string **custom**:

```
if (ipcom_strncmp(plink->devname, "custom", 6) == 0)
```

[use the custom serial device open procedure]

```
else
```

[follow the existing procedure in **ipcom_drv_ppp_open**]

If the open procedure generates an integer type file descriptor, the **plink->fd** member could be used to store it, otherwise a new structure member should be added to the **Ipcom_drv_ppp_link** structure, if required.

Input Operations

The function **ipcom_drv_ppp_inputd** (run as a separate task) handles input (read) operations for VxWorks serial devices.

If the custom serial device supports a select operation, that is, the ability to wait for activity on file descriptor, this function can be used with minimal changes, by simply adding the custom device input operation to the read function used for VxWorks serial devices, if the device name (**plink->devname**) indicates that the custom serial device is being used. The file descriptor associated with the custom serial device should be included in the select set.

If select is not supported, a means must be provided for closing a blocking read. See *Prerequisites*. Closing a blocking read must be added to the operation performed when a connection is closed. The input operation may need to clean up after such an interrupt has taken place.

Output Operations

The function **ipcom_drv_ppp_output** (run as a separate task) handles output (write) operations for VxWorks serial devices. This code would need to be augmented by comparing the name of the device (stored in **plink->devname**) for the custom device name, and, if matching, using the output procedure for the custom serial device.

Device Close

The function **ipcom_drv_ppp_exit** is called when the serial device needs to be closed. This happens when the ifconfig down command is entered, or the **IP_SIOCXCLOSE** ioctl has been called. If the custom serial device does not handle the select command, and the input operation is blocking, the custom serial device should interrupt the input operation in this function.

WNDRVR

Removing the Interface

Interface deletion is handled by the **IP_SIOCDESTROY** ioctl. If any special handling of custom serial devices is required before the interface is removed, it should be added to the section of code in the **IP_SIOCIFDESTROY** case section of **ipcom_drv_ppp_ioctl**. This section of code is executed when the ifconfig destroy command is entered, or the **IP_SIOCDESTROY** ioctl is used. Destroying an interface does not affect the value of the system variables that define the device name as described above.

Previous topic: PPP and Serial Interfaces on page 60
Next topic: Creating Interfaces at Initialization Time on page 66

4. Creating Interfaces at Initialization Time

At system boot up, if the Point-to-Point Protocol (PPP) component is included, the PPP driver code will create any interfaces defined by system variables.

The required system variables are defined in the file **ipppp_config.c**, as part of the static initialization of **ipppp_conf_sysvar**. The following line, for example, would create a PPP interface named **ppp5** using custom serial device name **custom0**:

```
{ "ipcom.if.ppp5.devname",  "custom0" }
```

Interfaces created this way can be used and destroyed the same way as interfaces created using the methods described in *Interface Creation* in Guidelines for Using PPP on Custom Serial Devices on page 63.

Previous topic: Guidelines for Using PPP on Custom Serial Devices on page 63
Next topic: PPPoE and Ethernet Interfaces on page 66

5. PPPoE and Ethernet Interfaces

Point-to-Point Protocol over Ethernet (PPPoE) interfaces are not created automatically. Each interface must be created individually and bound to an Ethernet interface.

To create an interface, use ifconfig. Interface names beginning with "pppoe" are automatically created as PPPoE interfaces. See the VxWorks 7 Network Stack Programmer's Guide for details about ifconfig. For example, the following will create PPPoE interface **pppoe0**.

```
# ifconfig pppoe0 create
```

To view the configuration, use the pppconfig -a command. For example:

```
# pppconfig -a
pppoe0  PPP
        inet 0.0.0.0  mask 255.255.255.255  peer 0.0.0.0
        [netif]  mtu=1492 flags=POINTOPOINT MULTICAST NOARP LINK1
        [ppp]    restart=3 max-failure=10 max-configure=10
max-terminate=2 max-authreq=3
              flags=noaccomp noaccm nopcomp

        [auth] username=<none> password=<none> peer=<none>
```

```
        [lcp] echo-interval=60  echo-failure=5
        [drv] ssn=0  baudrate=0  rcv-accm=0x0 snd-accm=0x0  mru=1500  mtu=1500
```

To bind a PPPoE interface to an Ethernet interface, use the pppconfig command or the **IP_SIOCXPPPSETIF** ioctl( ) request. See About PPP Shell Commands on page 37 and ioctl( ) Requests on page 53 for details. For example, the following will bind PPPoE interface **pppoe0** to Ethernet interface **eth1**.

```
# pppconfig pppoe0 pppoe-setif eth1
```

Previous topic: Creating Interfaces at Initialization Time on page 66
Next topic: PPP MP Bundles and Member Links on page 67

6. PPP MP Bundles and Member Links

An interface becomes a bundle when it negotiates Point-to-Point Protocol (PPP) Multilink with a peer that does not already have a bundle. If the peer already has a bundle, the interface becomes a member link of the bundle.

The first link established between PPP MP-enabled peers becomes the master link for the bundle, and all remaining member links are secondary. Links can be added and removed without terminating the bundle or otherwise interfering with communications (except for the impact on effective transfer rate through the bundle), but when the master link, the interface that first established the bundle, is taken down, all links are closed.

For example, the following commands enable PPP MP and create a Point-to-Point Protocol over Ethernet (PPPoE) interface.

The following command enables PPP MP:

```
# pppconfig mp-enable 1
```

The following command creates the **pppoe0** connection:

```
# ifconfig pppoe0 create
```

The following command assigns an Ethernet interface to the **pppoe0** connection:

```
# pppconfig pppoe0 pppoe-setif eth0
```

The following command brings up the **ppoe0** PPP MP-enabled interface:

```
# ifconfig pppoe0 up
```

If the interface succeeds in coming up and the peer agrees to multilink, **pppoe0** becomes a bundle, so far consisting of a single link. When a second PPPoE interface is created, it is joined to the pppoe0 connection to form a bundle, as shown with the following commands:

The following command creates the **pppoe1** connection:

```
# ifconfig pppoe1 create
```

The following command assigns an Ethernet interface to the **pppoe1** connection:

```
# pppconfig pppoe1 pppoe-setif eth0
```

The following command brings up the **ppoe1** PPP MP-enabled interface:

```
# ifconfig pppoe1 up
```

When the link comes up it is joined to the bundle, assuming the service request has not changed and the second PPPoE session starts up with the same peer, and the peer agrees to multilink on this link. So **pppoe0** will now be a bundle consisting of two links, and **pppoe1** will be a member interface of that bundle and represents one member link. Member interfaces do not have their own IP address; there is only one IP address for the bundle.

The following command causes the member link to go down and be removed from the bundle, but communication through the bundle continues uninterrupted:

```
# ifconfig pppoe1 down
```

However, the following command causes both links to be taken down and the bundle closed:

```
# ifconfig pppoe0 down
```

Previous topic: PPPoE and Ethernet Interfaces on page 66
Next topic: Authentication on page 68

7. Authentication

When Point-to-Point Protocol is running as a PPP or Point-to-Point Protocol over Ethernet (PPPoE) server with Password Authentication Protocol (PAP) or Challenge Handshake Authentication Protocol (CHAP) authentication, it authenticates peers with username and password combinations stored in a user database.

The user database is managed with API functions from the VxWorks user identification library (**userIdentLib**) and the key database is managed with API functions from the IPCOM key database library (**ipcom_key_db**). For more information, refer to the reference entries for the **userIdentLib** and **ipcom_key_db** libraries.

Previous topic: PPP MP Bundles and Member Links on page 67

8. PPP SHELL COMMANDS

1. pppconfig

The pppconfig shell command allows you to list and shutdown interfaces, display the Point-to-Point Protocol (PPP) configuration, and configure PPP parameters.

The multilink options appear only if the VxWorks source build (VSB) and VxWorks image project (VIP) have been built with Point-to-Point Protocol (PPP) Multilink Protocol (MP).

To get help for the pppconfig command, use the pppconfig command. For example:

```
# pppconfig
usage:
        List PPP interface(s):
             > pppconfig <<interface> | -a> [-debug]
        Show PPP configuration:
             > pppconfig show
        Shutdown PPP interface:
             > pppconfig <interface> shutdown
        Modify global PPP configuration option:
             > pppconfig <option> <value(s)>
        Modify interface-specific PPP configuration option:
             > pppconfig <ifname> <option> <value(s)>
        Modify user-specific PPP configuration option:
             > pppconfig -user <username> <option> <value(s)>
BASE options:
        runmode start - Bring UP PPP interface when attached
                        exit - Close interface if failed connection
                        passive - Passive mode
                        wincompat - Support PPP connections on Windows using null modem
        flags defaultroute - Add peer ipaddress as defaultroute
                  proxyarp - Add peer ipaddress as
proxy arp entry
        max-failure <nr> - Max # ConfNak
before sending ConfRej
        max-configure <nr> - Max # ConfReq retransmissions
        max-terminate <nr> - Max # TermReq retransmissions
        baudrate <nr> - Driver baudrate
LCP options:
        mru <size> - Maximum Receive Unit
        mtu <size> - Maximum Transmit Unit
        noaccomp <1|0> - Disable in/out address/control compression
        noaccm <1|0> - Do not send or ack ACCM ConfReq
        restart <u16> - Retransmission timeout in seconds
        echo-interval <sec> - LCP EchoReq output interval (seconds)
        echo-failure <nr> - Max LCP EchoReq failure before terminating
        nopcomp <1|0> - Enable/disable the negotiation of protocol compression
        accm-request <nr> - Set a non-default asynchronous control character map (ACCM)
AUTH options:
        auth auth - Require peer to authenticate
     noauth - Do not require peer to authenticate
     pap - Request peer PAP authentication
     chap - Request peer CHAP authentication
```

```
     refuse-pap - Do not agree to authenticate using PAP
     refuse-chap - Do not agree to authenticate using CHAP
        max-authreq <nr> - Max # auth retransmissions
        username <txt> - Username when logging in on peer
        password <txt> - Password when logging in on peer
IPCP options:
        addr <ipaddress> - Local IPv4 address
        dstaddr <ipaddress> - Destination (peer) IPv4 address)
        pool <poolname> - Choose pool
        setpool <poolname> <start ipaddress><end ipaddress> - Set Pool
PPPoE options:
        pppoe-server <1|0> - Enable/disable PPPoE server
        max-sessions <nr> - Maximum total PPPoE sessions
        max-eth-sessions <nr> - Maximum PPPoE sessions per interface
        pppoe-setif <ifname> - Set PPPoE client Ethernet interface
Multilink options:
        mp-enable <1|0> - Enable/disable
multilink protocol
        pppoe-links <nr> - Number of links to open (PPPoE only)
        mrru <nr> - Maximum Receive Reconstructed Unit
```

Base Options

All base options can be applied on a per-interface basis. No base option can be applied on a per-user basis.

runmode *mode* [","*mode*]*

where *mode* is start |passive | exit | wincompat

- If value is start, bring up PPP interface when attached.
- If value is exit, close interface if connection fails.
- If value is passive, attempt connection once when started.
- If value is wincompat, supports PPP connections to a Windows machine using a null modem.

See **PPP_RUNMODE** in the table PPP Configuration Parameters on page 18 and **PPP_IF_RUNMODE_LIST** in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command format is **sysvar set -o ipppp.runmode** *mode.*

flags defaultroute | **proxyarp**

- If value is **defaultroute**, add peer IP address as the default route.
- If value is **proxyarp**, add peer IP address as the proxy ARP entry.

See **PPP_FLAGS** in the table PPP Configuration Parameters on page 18 and **PPP_IF_FLAGS_LIST** in the table PPP Interface Configuration Parameters on page 26 The equivalent **sysvar** command format is **sysvar set -o ipppp.flags** *flag* .

max-failure *<nr>*

Max # ConfNak before sending ConfRej.

max-configure *<nr>*

Max # ConfReq retransmissions

max-terminate *<nr>*

Max # TermReq retransmissions

WNDRVR

baudrate *<nr>*

Driver baudrate. See **PPP_DEFAULT_BAUDRATE** in the table PPP Interface Configuration Parameters on page 26 and **PPP_IF_DEFAULT_BAUDRATE_LIST** in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command format is **sysvar set -o ipppp.baudrate** *nr* .

LCP Options

All LCP options can be applied on a per-interface basis. Only the echo-interval option can be set on a per-user basis.

mru *<size>*

Maximum Receive Unit. See **PPP_LCP_MRU** in the table PPP Interface Configuration Parameters on page 26 and **PPP_IF_LCP_MRU_LIST**in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command format is **sysvar set -o ipppp.lcp.mru** *nr* .

mtu *<size>*.

Maximum Transmit Unit. See **PPP_LCP_MTU** in the table PPP Interface Configuration Parameters on page 26 and **PPP_IF_LCP_MTU_LIST** in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command format is **sysvar set -o ipppp.lcp.mtu** *nr* .

noaccomp <**1**|**0**>

Disable in/out address/control compression

noaccm <**1**|**0**>

Do not send or ack ACCM ConfReq

restart *<u16>*

Retransmission timeout in seconds

echo-interval *<sec>*

LCP EchoReq output interval (seconds). See **PPP_LCP_ECHO_REQ_INTERVAL** in the table PPP Interface Configuration Parameters on page 26 and **PPP_IF_LCP_ECHO_REQ_INTERVAL_LIST** in the table PPP Interface Configuration Parameters on page 26 and **PPP_USERS_LCP_ECHO_REQ_INTERVAL_LIST** in the table PPP User Configuration Parameters on page 28. The equivalent **sysvar** command format is **sysvar set -o ipppp.lcp.echo_interval** *nr.*

echo-failure *<nr>*

Max LCP EchoReq failure before terminating. See **PPP_LCP_ECHO_REQ_FAILURE** in the table PPP Interface Configuration Parameters on page 26 and **PPP_IF_LCP_ECHO_REQ_FAILURE_LIST** in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command is **sysvar set -o ipppp.lcp.echo_failure** *nr.*

nopcomp *<arg>*

Controls the negotiation of protocol compression. PPP packets all have a protocol field in the header signifying the protocol of the encapsulated content. By default the field is 16 bits, but 8 bit codes can be negotiated by agreement of both sides, saving packet overhead. Protocol field compression is described in RFC 1661 section 6.5. The default behavior is to offer protocol field compression during link negotiation. The command format is as follows:

```
# pppconfig interface nopcomp arg
```

where *interface* is an interface such as ppp0, pppoe2, and so on, and *arg* is either 0 or 1; an argument of 1 will prevent the interface from negotiating protocol compression. A value of 0 results in the default behavior; nevertheless, an argument of either 0 or 1 must be provided.

accm-request *<non-default-accm>*

Allows user to enter a non-default asynchronous control character map(ACCM). The command format is as follows:

WNDRVR

```
# pppconfig interface accm-request non-default-accm
```

where *interface* is a PPP interface such as ppp0, and *non-default-accm* is a number that represents a 32-bit control character map. If the number starts with "0x" it is taken as a hex number, otherwise it is interpreted as a decimal number. The ACCM request goes into outgoing configuration requests and, if accepted by the peer, is used once link control protocol negotiation has completed. ACCMs are not used in PPPoE.

To determine if the requested ACCM was accepted by the peer, examine the interface with the **pppconfig** command after the link has come up:

```
# pppconfig interface
```

The negotiated ACCM is the "rcv-accm" in this listing. The ACCM negotiated for the peer is "snd-accm".

The ACCM request can also be configured in **ipppp_config.c** by adding a sysvar setting to the **ipppp_conf_sysvar** static initialization. For example to set the accm request for ppp0 to 0, add the following line before the closing "{IP_NULL, IP_NULL}":

```
{"ipcom.if.ppp0.ipppp.req_rcv_accm, "0"},
```

AUTH Options

All auth options can be applied on a per-interface basis. No auth option can be applied on a per-user basis.

auth auth | noauth | pap | chap | refuse-pap | refuse-chap

See PPP_AUTH_MODES in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_AUTH_MODES_LIST in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.auth *auth* .

- If value is auth, require peer to authenticate.
- If value is noauth, do not require peer to authenticate.
- If value is pap, request peer PAP authentication.
- If value is chap, request peer CHAP authentication.
- If value is refuse-pap, do not agree to authenticate using PAP.

If value is refuse-chap, do not agree to authenticate using CHAP.

max-authreq <*nr*>

Max # auth retransmissions

username <*txt*>

Username when logging in on peer. See PPP_USERNAME in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.username *nr* .

password <*txt*>

Password when logging in on peer. See PPP_PASSWD in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.password *nr* .

IPCP Options

All Internet Protocol Control Protocol (IPCP) options with the exception of setpool can be applied on a per-interface basis.

addr *<ipaddress>*

Local IPv4 address. See PPP_IPCP_IPV4_ADDRESS in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_IPCP_IPV4_ADDRESS_LIST in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.addr *ipv4-address*.

dstaddr *<ipaddress>*

Destination (peer) IPv4 address). See PPP_IPCP_PEER_IPV4_ADDRESS in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_IPCP_PEER_IPV4_ADDRESS_LIST in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.dstaddr *ipv4-address*.

pool *<poolname>*

Choose pool. See PPP_IPCP_PEER_IPV4_POOL_NAME in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command is sysvar set -o ipppp.ipcp.pool *pool-name*.

setpool *<poolname>* *<startIpaddress>* *<endIpaddress>*

Set Pool. See PPP_IPCP_PEER_IPV4_POOL_ADDRESS_POOL in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_IPCP_PEER_IPV4_POOL_ADDRESS_POOL_LIST in the table PPP Interface Configuration Parameters on page 26. The equivalent **sysvar** command format is sysvar set -o ipppp.ipcp.pool.default *startIpv4Address* **-** *endIpv4Address*. The hyphen between *startipv4Address* and *endipv4Address* is a literal.

dns-primary *<ipaddress>*

See PPP_IPCP_PRIMARY_DNS_ADDRESS in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_IPCP_PRIMARY_DNS_ADDRESS_LIST in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.primary_dns_address *ipv4-address*.

dns-secondary *<ipaddress>*

See PPP_IPCP_SECONDARY_DNS_ADDRESS in the table PPP Interface Configuration Parameters on page 26 and PPP_IF_IPCP_SECONDARY_DNS_ADDRESS_LIST in the table titled PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.secondary_dns_address *ipv4-address*.

wins-primary *<ipaddress>*

See PPP_IPCP_PRIMARY_NBNS_ADDRESS in the table titled PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.primary_nbns_address *ipv4-address*.

wins-secondary *<ipaddress>*

See PPP_IPCP_SECONDARY_NBNS_ADDRESS in the table PPP Interface Configuration Parameters on page 26. The equivalent sysvar command format is sysvar set -o ipppp.ipcp.secondary_nbns_address *ipv4-address*.

PPPoE Options

All PPPoE options with the exception of max-sessions can be applied on a per-interface basis. No PPPoE option can be applied on a per-user basis.

pppoe-server <**1**|**0**>

Enable/disable PPPoE server. See PPPOE_SERVER in the table PPPOE Configuration Parameters on page 29. The equivalent sysvar command format is sysvar set -o ipppp.pppoe.server *0*|*1*.

max-sessions *<nr>*

Maximum total PPPoE sessions. See PPPOE_MAX_SESSIONS in the table PPPOE Configuration Parameters on page 29. The equivalent **sysvar** command format is sysvar set -o ipppp.pppoe.max_sessions *nr*.

max-eth-sessions <*nr*>

Maximum PPPoE sessions per interface. See PPPOE_MAX_ETH_SESSIONS in the table PPPOE Configuration Parameters on page 29. The equivalent sysvar command format is sysvar set -o ipppp.pppoe.max_eth_sessions *nr*.

pppoe-setif <*ifname*>

Set PPPoE client Ethernet interface

Multilink Options

No multilink options can be applied on a per-interface basis. No multilink option can be applied on a per-user basis.

mp-enable <**1**|**0**>

Enable/disable multilink protocol. See PPPMP_ENABLE in the table PPP Multilink Protocol Configuration Parameters on page 31.

pppoe-links <*nr*>

Number of links to open (PPPoE only). See PPPMP_PPPOE_LINKS in the table PPP Multilink Protocol Configuration Parameters on page 31.

mrru <*nr*>

Maximum Receive Reconstructed Unit. See PPPMP_MRRU in the table PPP Multilink Protocol Configuration Parameters on page 31.

Additional Interface Options

drv-up

Establishes a link without establishing a PPP connection. Use to test link viability or test the operation of a serial driver. An example invocation is pppconfig *interface* **drv-up**.

drv-down

Tears down the link established using the drv-up option. An example invocation is pppconfig *interface* drv-down.